

Solving Concealed ILWE and its Application for Breaking Masked Dilithium

Simon Damm¹ , Asja Fischer¹ , Alexander May¹ ,
Soundes Marzougui^{2,4} , Leander Schwarz³, Henning Seidler³,
Jean-Pierre Seifert³ , Jonas Thietke¹ , and Vincent Quentin Ulitzsch³  *

¹ Faculty of Computer Science, Ruhr University Bochum, Germany
{simon.damm, asja.fischer, alex.may, jonas.thietke}@rub.de

² Deutsches Elektronen-Synchrotron, Germany
soundes.marzougui@desy.de

³ Technische Universität Berlin – SecT, Germany
{henning.seidler, vincent}@sect.tu-berlin.de
jean-pierre.seifert@tu-berlin.de
leander.schwarz@campus.tu-berlin.de

⁴ STMicroelectronics, Belgium

Abstract. Lattice-based signatures like Dilithium (ML-DSA) prove knowledge of a secret key $\mathbf{s} \in \mathbb{Z}^n$ by using Integer LWE (ILWE) samples $z = \langle \mathbf{c}, \mathbf{s} \rangle + y$, for some known hash value $\mathbf{c} \in \mathbb{Z}^n$ of the message and unknown error y . Rejection sampling guarantees zero-knowledge, which makes the ILWE problem, that asks to recover $\mathbf{s}$ from many z 's, unsolvable.

Side-channel attacks partially recover y , thereby obtaining more informative samples resulting in a—potentially tractable—ILWE problem. The standard method to solve the resulting problem is *Ordinary Least Squares* (OLS), which requires independence of y from $\langle \mathbf{c}, \mathbf{s} \rangle$ —an assumption that is violated by zero-knowledge samples.

We present efficient algorithms for a variant of the ILWE problem that was not addressed in prior work, which we coin *Concealed ILWE (CILWE)*. In this variant, only a fraction of the ILWE samples is zero-knowledge. We call this fraction the *concealment rate*. This ILWE variant naturally occurs in side-channel attacks on lattice-based signatures. A case in point are profiling side-channel attacks on Dilithium implementations that classify whether $y = 0$. This gives rise to either zero-error ILWE samples $z = \langle \mathbf{c}, \mathbf{s} \rangle$ with $y = 0$ (in case of correct classification), or ordinary zero-knowledge ILWE samples (in case of misclassification).

As we show, OLS is not practical for CILWE instances, as it requires a prohibitively large amount of samples for even small (under 10%) concealment rates. A known integer linear programming-based approach can solve some CILWE instances, but suffers from two short-comings. First, it lacks provable efficiency guarantees, as ILP is NP-hard in the worst case. Second, it does not utilize *small, independent error y* samples, that could occur in addition to zero-knowledge samples.

* Authors are listed in alphabetical order.

We introduce two statistical regression methods to cryptanalysis, *Huber* and *Cauchy* regression. They are both efficient and can handle instances with all three types of samples. At the same time, they are capable of handling high concealment rates, up to 90% in practical experiments. While Huber regression comes with theoretically appealing correctness guarantees, Cauchy regression performs best in practice.

We use this efficacy to execute a novel profiling attack against a masked Dilithium implementation. The resulting ILWE instances suffer from both concealment and small, independent errors. As such, neither OLS nor ILP can recover the secret key. Cauchy regression, however, allows us to recover the secret key in under two minutes for all NIST security levels.

Keywords: Side-Channel Analysis, Dilithium, Fiat-Shamir with Aborts, Integer LWE, Regression

1 Introduction

Lattice-based signature schemes based on the hardness of the Learning with Errors (LWE) problem constitute a major class of post-quantum primitives. Among the various design frameworks for lattice-based signatures, the Fiat-Shamir with Aborts paradigm – introduced by Lyubashevsky in [Lyu09] – stands out for allowing efficient and relatively simple schemes.

A prominent instantiation of this paradigm is Dilithium [BDK⁺21] which constructs its signature by computing for some known challenge $\mathbf{c}_i \in \mathbb{Z}^n$ —a randomized hash value of the signed message— an Integer LWE (ILWE) sample $z_i = \langle \mathbf{c}_i, \mathbf{s} \rangle + y_i$, where y_i is some uniformly random (secret) randomness. However, $(z_i, \mathbf{c}_i)$ is only output as a valid signature if it does not leak information about $\mathbf{s}$, thereby guaranteeing the zero-knowledge property of the *Fiat-Shamir with Aborts* technique. This process is called *rejection sampling*, and the construction of ILWE samples is repeated with fresh y_i and rerandomised $\mathbf{c}_i$, until z_i satisfies the zero-knowledge property. An ILWE sample $(z_i, \mathbf{c}_i)$ is called *zero-knowledge* if it passed the rejection sampling process of Fiat-Shamir with Aborts. Notice that zero-knowledge ILWE samples information-theoretically hide $\mathbf{s}$ because every possible secret could have generated a given sample set. Thus, zero-knowledge ILWE samples lead to an unsolvable ILWE problem, and thereby to a secure signature scheme.

However, in practical implementations, this zero-knowledge guarantee might degrade due to physical leakage on the error y_i . We highlight that implementations for both Dilithium as well as ML-DSA as specified by FIPS 204 [BDK⁺21, Nat24] can suffer from this issue.

Ordinary Least Squares attacks. The first thorough cryptanalysis of ILWE and its formal definition was given by Bootle et al. in [BDE⁺18] by targeting the BLISS signature scheme [DILL13]. The authors obtained ILWE samples from

a side channel information on BLISS. Bootle et al. proved, that *Ordinary Least Squares* Regression (OLS) finds a solution given certain (subgaussianess and independence) constraints on the distributions of $\mathbf{c}$, z and y . Different leakage points on the hardened BLISS implementation named GALACTICS [BBE⁺19] were attacked and exploited via OLS by [MWG⁺22].

In [LZS⁺20], Liu et al. exploited a side channel on the reference implementation of Dilithium [BDK⁺], where they assume a one-bit leakage in one of the coefficients of y . The authors exploited the fact that y is much larger than $\langle \mathbf{c}, \mathbf{s} \rangle$, to elevate this one-bit leakage into additional information about y through observing the most significant bits of z . Thus, Liu et al. are able to break the zero-knowledge property and obtain an ILWE distribution, which fulfils conditions similar to those of [BDE⁺18]. Henceforth, they use OLS on the resulting observations to recover the secret key.

Later on, Wang et al. extend the underlying ILWE problem to errors that depend linearly on the secret key [WLX⁺21]. Due to a well-behaved generalisation of the error distributions, they are still able to solve the resulting system of equations with OLS. This new theoretical approach was used by Qiao et al. in [QLZ⁺22] in order to exploit a similar side-channel attack even on masked versions of Dilithium. The efficiency of the attack in [LZS⁺20] was improved later in [DKM⁺25], where the authors provide a novel transformation of the obtained equations and a more fine-grained analysis in order to lower the sample complexity of the attack both practically and theoretically.

Other side-channels on Dilithium. An early side-channel attack against Dilithium was carried out by Ravi et al. in [RJH⁺18]. This attack targets the multiplication of the challenge c with Dilithium secret vector $\mathbf{s}_1$. The work shows that recovering only $\mathbf{s}_1$ is sufficient to generate valid signatures that pass verification by the victim’s public key (instead of both $(\mathbf{s}_1, \mathbf{s}_2)$).

Subsequent attacks have focused predominantly on recovering $\mathbf{s}_1$. A common strategy is to obtain leakage on randomness $\mathbf{y}$. For instance, Ulitzsch et al. [UMTS24] presented a power side-channel attack that exploits leakage on the zero coefficients of the randomness $\mathbf{y}$. The authors gather side-channel observations into a noisy equation system and recover $\mathbf{s}_1$ by formulating an integer linear program (ILP) that maximizes the number of satisfied equations. Similarly, Berzati et al. [BVC⁺23] proposed a template attack, that also aimed to identify whether each coefficient of $\mathbf{y}$ is zero or not. By accurately classifying these coefficients and applying linear algebra techniques, Berzati et al. achieve key recovery. A recent concurrent work by Hermelink et al. [HNP25] demonstrates a template-based power side-channel attack on a masked Dilithium (ML-DSA) implementation proposed by Coron et al. [CGL⁺24].

In [SLKG22], Steffen et al. investigated hardware implementations and showed that the Number-Theoretic Transform (NTT) in Dilithium remains vulnerable to profiled Simple Power Analysis and Correlation Power Analysis.

Our Concealed ILWE Model. In general, side-channel based cryptanalysis tries to bypass the zero-knowledge property of ILWE samples $z_i = \langle \mathbf{c}_i, \mathbf{s} \rangle + y$, usually

by leaking some information about the randomness y . Notice that $\mathcal{O}(n)$ ILWE samples with $y = 0$ are sufficient to solve for $\mathbf{s}$ using Gaussian elimination.

Therefore, a straightforward side-channel profiling model may classify ILWE samples into those having *zero-error* $y = 0$, or non-zero error $y \neq 0$. If we had a perfect profiling model with 100% classification accuracy, this would identify *zero-error* ILWE samples and thus lead to a simple Gaussian elimination attack.

However, in practice, we usually obtain significantly lower accuracy of side-channel classification. This means that we misclassify some zero-knowledge ILWE samples with $y \neq 0$ falsely as *zero-error* ILWE samples, leaving a cryptanalyst with a mixture of zero-knowledge (bad) and zero-error (good) ILWE samples.

We call such a mixture of ILWE samples *Concealed ILWE* (CILWE), since the zero-knowledge samples conceal the information about $\mathbf{s}$. The fraction of zero-knowledge samples among all samples is called the concealment rate p of CILWE.

High accuracy does not automatically imply small concealment rate. In practice, side-channel attacks can give rise to CILWE instances with high concealment rate, as the event that we classify might be unlikely. As a case in point, notice that Dilithium’s randomness y is an 18-bit integer chosen uniformly at random. Our side-channel attack on Dilithium filters to y chosen from the set $\{-78, \dots, 78\}$. We target the event that y has zero Hamming weight $w(y) = 0$, which happens with probability $\mathbb{P}(w(y) = 0) = \mathbb{P}(y = 0) = \frac{1}{157}$.

Now, assume that our classifier for the event $w(y) = 0$ has a very high true positive rate of 99% and a very low false positive rate of 1%. We use Bayes’ theorem, and still obtain a concealment rate of

$$p = \frac{0.01 \mathbb{P}(y \neq 0)}{0.99 \mathbb{P}(y = 0) + 0.01 \mathbb{P}(y \neq 0)} > 0.6.$$

In other words, since we target an unlikely event, even a high accuracy does not prevent us from obtaining a majority of over 60% zero-knowledge ILWE samples that conceal our sample set.

Why OLS regression and ILP fail. Here, we briefly explain why Ordinary Least Squares (OLS) regression is not suited for solving CILWE.

On the theoretical side, OLS only works under the requirement that the error y is centred at zero and independent of the secret $\mathbf{s}$ [SL03]. Intuitively, this requirement guarantees that the error is spread out evenly around the regression line $\langle \mathbf{c}, \mathbf{s} \rangle$ for $\mathbf{s}$. Figure 1 illustrates a low dimensional example, with y sampled uniformly from $\{-2, -1, \dots, 2\}$ and $\mathbf{s} \in \mathbb{Z}$.

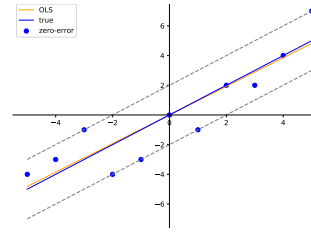


Fig. 1: OLS works: independent error y

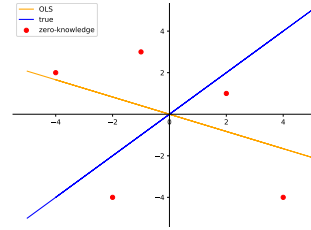


Fig. 2: OLS must fail: zero-knowledge ILWE

However, OLS' independence requirement does not hold for zero-knowledge samples. Recall that a zero-knowledge ILWE sample $(z, \mathbf{c})$ with $z = \langle \mathbf{c}, \mathbf{s} \rangle + y$ does not reveal anything about $\mathbf{s}$. This immediately implies that y cannot be independent of $\mathbf{s}$. Namely, positive values of y positively correlate with negative values of $\langle \mathbf{c}, \mathbf{s} \rangle$, and vice versa; see also Figure 2. Since by the zero-knowledge property we obtain no information about the true regression line, OLS (or any other algorithm) cannot succeed.

In Figure 3 we illustrate a mixture of zero-knowledge and zero-error ILWE samples, the CILWE case. Namely, the blue zero-error samples all lie on the true regression line, and the yellow zero-knowledge ILWE samples are randomly distributed in space, independent of the regression line. As opposed to Figure 2, we see that the zero-error samples rotate the OLS solution closer to the true regression line. However, especially the outlier points at positions $(-4, 2)$ and $(4, -4)$ force OLS regression to fail.

As an alternative method, one may apply Integer Linear Programming (ILP), a discrete optimisation tool that has been successfully applied in the context of other ILWE problems [UMTS24]. In general, solving Integer Linear Programming is known to be NP hard. Still, ILP may perform well on some practical ILWE instances.

This efficiency on some practical ILWE instances enabled the use of ILP in hardware attacks on Fiat–Shamir with Aborts attacks. However, as we will show, ILP fails to handle instances with high concealment rate. In addition, the ILP formulation given by prior work has the drawback that it requires that some ILWE samples have exactly zero error. Regression-based methods can, in practice, also handle ILWE samples with a small error, as long as it is independent from $\langle \mathbf{c}, \mathbf{s} \rangle$.

In contrast to ILP, our new methods come with provable run-time and correctness guarantees. We demonstrate experimentally that the newly introduced regression methods outperform ILP, both on synthetic instances and instances encountered in side-channel attacks on masked Dilithium.

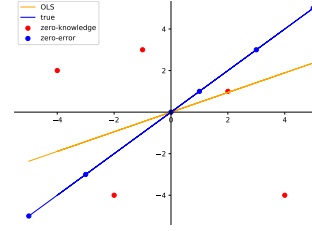


Fig. 3: OLS still fails: CILWE

Our Results.

Introduction of the CILWE problem. For some secret vector $\mathbf{s} \in \mathbb{Z}^n$ our CILWE is composed of samples $(z_i, \mathbf{c}_i) \in \mathbb{Z} \times \mathbb{Z}^n$ with $z_i = \langle \mathbf{c}_i, \mathbf{s} \rangle + y$, where either $y = 0$ (zero-error case), or $(\mathbf{c}_i, z_i)$ reveal nothing about $\mathbf{s}$ (zero-knowledge case). The percentage of zero-knowledge samples among all samples is called its concealment rate p .

Huber regression, a theoretically sound solution. We introduce different statistical regression methods, novel to cryptanalysis, for solving CILWE.

As we argued, OLS is not suited for solving CILWE. In a nutshell, OLS estimates a secret that minimises the average distance in the ℓ_2 -norm of every

sample points to the linear regression line. For a single sample, we call this difference a residual, and the minimised function the loss function. This implies that outlier points far from the desired line – as introduced by our zero-knowledge samples – largely influence the regression estimate. However, ideally, we would like to ignore the concealing zero-knowledge samples and instead only use the zero-error samples.

To this end, we introduce *Huber regression* that moderates the effect of zero-knowledge samples. It finds an estimate that minimises a different loss function. Huber regression [Hub64] uses the ℓ_2 -norm only inside a strip of small width 2δ , whereas outside the strip Huber regression utilises the ℓ_1 -norm to dampen the influence of zero-knowledge ILWE samples; see also Figure 4.

We show that Huber regression theoretically succeeds to solve CILWE (which is not possible using OLS), thereby proving for the Dilithium case a concrete lower number of required samples as a function of the concealment rate p .

Cauchy regression, a highly practical solution. We introduce Cauchy regression [Moo77] that further dampens the effect of outliers from the regression line by using only a logarithmic loss for all sample points (see Figure 4). Intuitively, a small loss on zero-knowledge samples limits their influence on the estimate. Unfortunately, the Cauchy loss is not convex, and therefore we cannot give theoretical convergence guarantees to a global optimum [HR09, BV04]. However, we demonstrate by extensive experimental data that Cauchy regression is superior to all other methods in terms of required number of samples. For example, we are able to solve CILWE with concealment rate $p = 0.75$ with only 3,400 samples in 10 seconds, whereas Huber requires 14,500 samples and both OLS and ILP do not succeed.

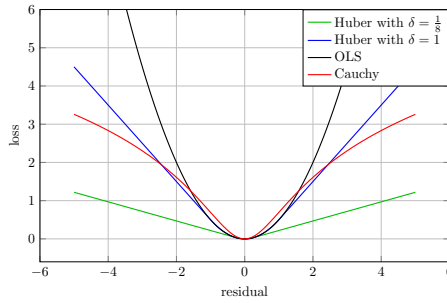


Fig. 4: Loss functions of different regressions.

A novel simple side-channel attack on masked Dilithium. The regression algorithms presented in this paper enable a novel profiling side-channel attack against the masked Dilithium implementation recently proposed by Coron et al. [CGTZ23]. We exploit a micro-architectural leakage to recover information about $\mathbf{y}$ through a power side-channel. A simple machine-learning classifier allows an attacker to distinguish whether a given coefficient is positive. The resulting instances give rise to ILWE-instances with zero-error samples, small independent-error samples, and zero-knowledge samples. As such, neither an OLS nor an ILP approach manage to recover the secret key - Cauchy regression allows for full secret key recovery within minutes. We note that our machine learning-assisted side-channel attack requires more signatures in the attack phase but is simpler and requires less extensive profiling than the concurrently

proposed templating attack of [HNP25]. Furthermore, we perform our proof-of-concept attack against all NIST security levels, while prior work demonstrated attacks against masked Dilithium NIST security level 2.

Improvement over previous results. Through extensive simulations, we show that Huber and Cauchy regression outperform an ILP-based approach presented in prior work [UMTS24] in terms of required signatures, ILWE samples and running time. For example, for security level 2, [UMTS24] require 800,000 Dilithium signatures for a false positive rate of 0.019, whereas we require only 550,000. Furthermore, Huber and Cauchy regression allow for much higher concealment rate than Integer Linear Programming or OLS.

Implementations. Our Dilithium side channel data and the corresponding Huber and Cauchy regression algorithms are available at the following Github link: https://github.com/fgsect/concealed_ilwe. All experiments can be reproduced using the artifact provided with this paper.

Organization of the paper. Section 3 introduces the Concealed ILWE Model. As a case in point, we show that machine-learning-based Dilithium side-channel attacks fit well into the CILWE setting. In Section 4 we introduce Huber and Cauchy regression, and prove Huber’s correctness. We provide numerous experimental results that demonstrate their efficiency and analyse the required number of ILWE samples as a function of the concealment rate p in Section 5. We use these regression methods for efficiently extracting Dilithium secret keys. In Section 6 we demonstrate a profiling power side-channel attack against masked Dilithium, in which we recover the secret key through Cauchy regression. Section 7 provides a theoretical analysis under which conditions Huber regression provably solves CILWE.

2 Background

2.1 Notation

Throughout this paper, lowercase bold letters will refer to vectors, and uppercase bold letters will refer to matrices. Scalars in $\mathbb{Z}$ or a discrete group will be denoted by lowercase italicised letters. We will denote by R_q the ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. For a given vector of polynomials $\mathbf{y} \in R_q^\ell$, we will denote by $\mathbf{y}_{i,j}$ the j ’th coefficient of the i ’th polynomial of $\mathbf{y}$.

2.2 Fiat–Shamir with Aborts Signature Schemes

Lyubashevsky [Lyu09] describes a construction for efficient lattice-based signature schemes based on the Fiat-Shamir transform [FS87]. We give a high-level description of the signing process, focused on ILWE samples. To this end, we

regard the vectors and polynomials coefficient-wise, thus obtaining statements over $\mathbb{Z}$.

The signer wants to sign a message $\mathbf{m} \in \{0, 1\}^*$, using a private key $\mathbf{s} \in \mathbb{Z}^n$. To do so, the signer first samples some randomness $y \in \mathbb{Z}$ from a bounded interval, then uses a collision resistant hash function H to hash $\mathbf{m}$ together with y into a vector in $\mathbf{c} \in \mathbb{Z}^n$ (the challenge) and then computes $z = \langle \mathbf{s}, \mathbf{c} \rangle + y$. Crucially, the role of y is to *hide* the precise value of $\langle \mathbf{s}, \mathbf{c} \rangle$. We repeat this process until z falls into a predefined range, thus ensuring that it does not leak any information about the secret $\mathbf{s}$. Through this *rejection sampling* step, the signer obtains a zero-knowledge signature $(z, \mathbf{c})$. For details, we refer to [Lyu09].

Input: $\mathbf{s} \in \mathbb{Z}^n$, message $\mathbf{m} \in \{0, 1\}^*$

Output: Zero-knowledge ILWE sample $(z, \mathbf{c})$

```

1 repeat
2   | Sample  $y \in \mathbb{Z}$  at random uniformly from a bounded interval
3   | Compute  $\mathbf{c} = H(\mathbf{m}, y) \in \mathbb{Z}^n$ 
4   | Compute  $z = \langle \mathbf{s}, \mathbf{c} \rangle + y$ 
5 until  $(z, \mathbf{c})$  is zero-knowledge

```

Algorithm 1: High-Level Description: Fiat Shamir with Aborts Signature

2.3 The Dilithium Signature Scheme

The Dilithium signature scheme is based on the Fiat–Shamir with Aborts structure [Lyu09]. Its security is tied to the hardness of the Module Learning with Errors and Module Short Integer Solution problems. In summary, Dilithium follows the above-described Fiat–Shamir with Aborts paradigm. It carries out its operations over the ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. The Dilithium secret key consists of two vectors of small polynomials $\mathbf{s}_1 \in R_q^\ell$ and $\mathbf{s}_2 \in R_q^k$, with each coefficient being chosen uniformly at random from a small range $\{-\eta, \dots, \eta\}$. The public key consists of a matrix $\mathbf{A} \in R_q^{k \times \ell}$, generated uniformly at random, and a vector $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$.

To sign a message $\mathbf{m}$, the signer takes the following steps. The signer first generates the randomness $\mathbf{y} \in R_q^\ell$ by sampling $\ell \cdot n$ uniformly random coefficients from the range $\{-\gamma_1 + 1, \dots, \gamma_1 - 1\}$. Next, the message $\mathbf{m}$ together with $\mathbf{A}\mathbf{y}$ is hashed to obtain a sparse polynomial $c = H(\mathbf{m}, \mathbf{A}\mathbf{y}) \in B_\tau$, where by B_τ we denote the ball with the set of elements of R that have τ coefficients which are either -1 or 1 and the rest are 0 . The signer then computes $\mathbf{z} = c\mathbf{s}_1 + \mathbf{y}$. The signer reveals $\mathbf{z}$ if it fulfils a certain condition, which mostly states $\mathbf{z}$'s coefficients fall into a small interval, and thereby ensures that $\mathbf{z}$'s coefficients do not reveal information about $\mathbf{s}_1$. Otherwise, the signer repeats the process with fresh $\mathbf{y}$. Algorithm 2 summarises this signing process. The parameters $k, \ell, \gamma_1, \gamma_2, \omega, \tau, \beta$ are dependent on Dilithium's security level.

In practice, the Dilithium signature algorithm employs further optimisations (which also involve $\mathbf{s}_2$) in the signing process, which we left out because they are not relevant to this paper. However, we highlight that knowledge of $\mathbf{s}_1$ is sufficient to sign arbitrary messages [GBP18, RJH⁺18]. For a more detailed description, we refer to the Dilithium specification [BDK⁺21].


```

1 Gen
2    $\mathbf{A} \leftarrow R_q^{k \times \ell}$ 
3    $(\mathbf{s}_1, \mathbf{s}_2) \leftarrow S_\eta^\ell \times S_\eta^k$ 
4    $\mathbf{t} := \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$ 
5   return  $(pk = (\mathbf{A}, \mathbf{t}), sk = (\mathbf{A}, \mathbf{t}, \mathbf{s}_1, \mathbf{s}_2))$ 
6 Sign  $(sk, M)$ 
7    $\mathbf{z} := \perp$ 
8   repeat
9      $\mathbf{y} \leftarrow S_{\gamma_1-1}^\ell$ 
10     $\mathbf{c} \in B_r := H(M \parallel \mathbf{A}\mathbf{y})$ 
11     $\mathbf{z} := \mathbf{c}\mathbf{s}_1 + \mathbf{y}$ 
12  until  $(\mathbf{z}, \mathbf{c})$  is zero-knowledge
13  return  $\sigma = (\mathbf{z}, \mathbf{c})$ 

```

Algorithm 2: Simplified description of Dilithium signature scheme. By S_η we denote the set $S_\eta = \{w \in R : \|w\|_\infty \leq \eta\}$, where $\|v\|_\infty = \max_i |v_i|$.

m	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Table 1: Confusion matrix

2.4 Classification Metrics

In this paper, we consider side channel attacks based on machine learning classifiers. We briefly explain which metrics we use to describe the quality of a binary classifier. The general task of such a classifier is to distinguish whether some element is part of the *positive class* or not —e.g. whether $y = 0$.

The basis of all quality metrics is the *confusion matrix* (Table 1). It shows how to call each possible classification result given the actual class.

The probability that an actual positive value gets classified as positive is called *true positive rate* (TPR). Likewise, the probability that an actual negative value gets classified as positive is called *false positive rate* (FPR).

A generic measurement is *accuracy*, which is defined as correct predictions over all classified samples, i.e. $(TP + TN)/m$, where m is the number of samples. Since, for our use case, we discard all negative classifications, we focus on *precision* $TP/(TP + FP)$.

3 ILWE and Concealed ILWE

In this section, we introduce the Concealed ILWE problem. We start by defining Integer LWE (ILWE) samples.

Definition 3.1 (ILWE samples). We call $(z_i, \mathbf{c}_i) \in \mathbb{Z} \times \mathbb{Z}^n$ ILWE samples for $\mathbf{s} \in \mathbb{Z}^n$ if $z_i = \langle \mathbf{c}_i, \mathbf{s} \rangle + y_i$ for some (unknown) randomness $y_i \in \mathbb{Z}$. We call y_i the error of a given ILWE sample and $\mathbf{s}$ the secret. We call ILWE

samples $(z_i, \mathbf{c}_i)$ zero-knowledge samples if y_i is drawn from a distribution that information-theoretically hides $\mathbf{s}$. ILWE samples $(z_i, \mathbf{c}_i)$ for which y_i is drawn from a distribution that always returns zero are called zero-error samples.

In this paper, we focus on a variant of ILWE where some of the samples are zero-knowledge, while others are zero-error.

Definition 3.2 (Concealed ILWE). Let $p \in [0, 1]$ and let $\mathbf{s}$ be an ILWE secret. In CILWE, we are given an oracle that outputs with probability p a zero-knowledge ILWE sample $(z, \mathbf{c})$, and with probability $(1 - p)$ a zero-error ILWE sample $(z, \mathbf{c})$. We call the parameter p the concealment rate.

Remark 3.3. In practice, regression methods can also recover the secret $\mathbf{s}$ in a setting where the samples are a mixture of zero-knowledge samples and samples with small, independent error y_i . We focus on the case of $y_i = 0$ for simplicity.

Practical motivation for the CILWE model. As explained in Section 1, various hardware attacks on Fiat–Shamir with Aborts signature schemes give rise to CILWE instances [UMTS24, UMB⁺23, KPLG24]. A Dilithium signature $(\mathbf{z}, c) \in (R_q^\ell \times R_q)$ gives rise to multiple (zero-knowledge) ILWE samples by regarding the vectors and polynomials coefficient-wise. Side-channel attacks classify whether a given coefficient $\mathbf{y}_{i,j}$ is positive. In case of a correct classification, we obtain a zero-error sample. In case of an incorrect classification, we obtain an ILWE sample with a potentially unknown error distribution, which CILWE pessimistically models as a zero-knowledge sample. Fault injection attacks set certain (unknown) coefficients $\mathbf{y}_{i,j}$ to 0, which is perfectly modelled by CILWE.

Hardware attacks on Dilithium can furthermore apply a *filtering* optimisation when deriving CILWE instances from signatures. Observe that for Dilithium ILWE samples, if $y_i = 0$, then z_i is a sum of τ random variables in $\pm\eta$ and by a central limit theorem argument, $z \sim \mathcal{N}(0, \sigma^2)$ with $\sigma^2 = \frac{(2\eta)^2 - 1}{12} \tau$ [UMTS24]. Therefore, large values of $|z_i|$ are unlikely. Using this observation, attackers can filter out samples for which $|z_i|$ exceeds a certain threshold, reducing the number of zero-knowledge samples at the expense of also ignoring a small number of zero-error samples.

3.1 Solving ILWE Through Regression

A large body of prior work has suggested to use ordinary least-squares (OLS) to recover the secret $\mathbf{s}$ from a given set of ILWE samples [BDE⁺18, LZS⁺20, DKM⁺25, WLX⁺21, QLZ⁺22, MWG⁺22]. We now introduce terminology to generalize regression-based algorithms that attempt to recover the ILWE secret from a given set of ILWE samples. On a high level, regression-based algorithms always follow Algorithm 3. First, the attacker uses a regression algorithm $\mathcal{A}$, called *estimator*, which, given a set of ILWE samples, computes an estimate $\hat{\mathbf{s}} \in \mathbb{R}^n$. Then, the attacker rounds each coefficient in $\hat{\mathbf{s}}$ to the nearest integer to obtain the ILWE secret $\mathbf{s}$. We call $\mathcal{A}$ *efficient* if it runs in polynomial time with respect to the number of samples and the dimension of the secret n . We call an

estimator $\mathcal{A}$ *sound* with respect to a given distribution of the ILWE samples if its rounded estimate $\hat{\mathbf{s}}$ matches the ILWE secret with high probability when given a number of samples polynomial in n and $(1-p)^{-1}$, where p is the concealment rate and the probability is over the drawn samples.

Input: ILWE samples $(z_i, \mathbf{c}_i) \in \mathbb{Z} \times \mathbb{Z}^n$

Output: LWE secret key $\mathbf{s} \in \mathbb{Z}^n$

- 1 Use estimator $\mathcal{A}$ to compute estimate $\hat{\mathbf{s}} \in \mathbb{R}^n$ from the ILWE samples.
- 2 Round the estimate to the nearest integer $\mathbf{s} = \lceil \hat{\mathbf{s}} \rceil \in \mathbb{Z}^n$.
- 3 **return** $\mathbf{s}$

Algorithm 3: High-Level Linear Regression Attack on ILWE

Limitations of prior work. The previously mentioned estimators, OLS and ILP, are both limited in their ability to solve CILWE.

Bootle et al. [BDE⁺18] prove that OLS is a sound estimator for ILWE samples under certain assumptions on their distributions. Trivially, it is also efficient [SL03]. However, the soundness of OLS is based on the assumption that each error y is statistically independent of the secret $\mathbf{s}$ and $\mathbf{c}$. This assumption is not fulfilled by CILWE instances. For zero-knowledge ILWE samples, the error y_i is not statistically independent of $\mathbf{s}$, thus violating the assumption of OLS. Our experiments verify that OLS fails to solve the instances arising in our setting, even for a low concealment rate and a huge number of samples.

The ILP-based estimator proposed by Ulitzsch et al. [UMTS24] is sound, but not efficient. This paper aims to fill this gap by utilizing estimators that are both efficient and sound to solve CILWE instances.

4 Huber and Cauchy Based Estimators for CILWE

In this Section, we introduce the *Huber* and *Cauchy* regression methods as sound and efficient estimators for CILWE instances. For both regression methods, we provide extensive simulations that indicate soundness and efficiency (Section 5.1). For Huber, we also prove soundness and efficiency in Section 7 for the case of CILWE instances arising from Dilithium.

4.1 Dampening the Effect of Outliers

Huber and Cauchy Regression are part of a wide class of *M estimators*, which are extensively studied in the statistics literature [HR09, MMYSB19].

For ILWE samples $(z_i, \mathbf{c}_i)$ regression methods try to find an $\mathbf{s}'$ that minimizes the distances of all z_i from the regression line through the points $(\mathbf{c}_i, \langle \mathbf{c}_i, \mathbf{s}' \rangle)$. These distances are called *residuals*, formally defined as follows.

Definition 4.1 (Residual, Loss function, Estimate).

Let $(z_i, \mathbf{c}_i)$ for $i \in \{1, \dots, m\}$ be ILWE samples, and let $\mathbf{s}' \in \mathbb{R}^n$ be an approximation of the ILWE solution $\mathbf{s} \in \mathbb{Z}^n$. We call $r_i(\mathbf{s}') := z_i - \langle \mathbf{c}_i, \mathbf{s}' \rangle$ the residual of $(z_i, \mathbf{c}_i)$ with respect to $\mathbf{s}'$. We define a loss function $\rho : \mathbb{R} \mapsto \mathbb{R}$ as a function which is non-negative, symmetric at the origin, and twice differentiable. We define the empirical loss function as its average on m given samples, i.e.

$$L_m(\mathbf{s}') = \frac{1}{m} \sum_{i=1}^m \rho(r_i(\mathbf{s}')) .$$

By the estimate $\hat{\mathbf{s}} \in \mathbb{R}^n$ we denote the minimiser of the empirical loss function, i.e., $\hat{\mathbf{s}} = \arg \min_{\mathbf{s}'} L_m(\mathbf{s}')$.

We obtain OLS regression by choosing in Definition 4.1 the loss function $\rho(x) = \frac{1}{2}x^2$. Notice that such a quadratic loss function gives overly large influence to large residuals, making it improper for our CILWE setting. Therefore, we study in the following loss functions that dampen the effect of large residuals.

4.2 Huber and Cauchy loss function

Informally, the Huber loss function uses a function that is quadratic inside a strip of width 2δ , where $\delta > 0$ is called Huber parameter, and linear outside. The following definition combines the quadratic and the linear part in a way that makes the loss function differentiable.

Definition 4.2 (Huber loss [Hub64]). We define the Huber loss function for some Huber parameter $\delta > 0$ via the loss function

$$\rho_\delta(r) = \begin{cases} \frac{1}{2}r^2 & \text{if } |r| \leq \delta \\ \delta(|r| - \frac{1}{2}\delta) & \text{if } |r| > \delta \end{cases} .$$

The Huber loss function [Hub64] for Huber parameters $\delta = \frac{1}{8}$ and $\delta = 1$ is depicted in Figure 4. As opposed to OLS' quadratic loss function, one already observes the dampening effect for large residuals by the linearity of the function outside the 2δ -range.

Further dampening is achieved by choosing a logarithmic loss function. This is done for Cauchy regression (see Figure 4).

Definition 4.3 (Cauchy loss [Moo77]). The Cauchy loss function is defined via the loss function $\rho(r) = \ln(1 + r^2)$.

We use a dampening loss function to reduce the influence of samples with large errors. However, since large residuals depend on our current approximation $\mathbf{s}'$ rather than the true $\mathbf{s}$, we cannot determine if they correspond to actual large errors. Therefore, it is beneficial to consider all samples.

Both estimators can also handle small, independent errors in the samples as they inherit the properties of OLS in a region around zero. We can therefore apply these in settings where we encounter both zero-error, zero-knowledge, and small, independent error samples.

4.3 Efficient Estimate Computation for Huber and Cauchy

In this section, we describe how to compute estimates $\hat{\mathbf{s}}$ for the Huber loss function (Definition 4.2) and Cauchy loss function (Definition 4.3).

We first revisit the computation of OLS. For samples $(z_i, \mathbf{c}_i)$ aggregated into a matrix $\mathbf{C}$ and a vector $\mathbf{z}$, OLS minimises the sum of squared residuals, given by

$$L_m(\mathbf{s}') = \frac{1}{m} \sum_{i=0}^m r_i(\mathbf{s}')^2 = \frac{1}{m} \left\| \mathbf{C}^T \mathbf{s}' - \mathbf{z} \right\|_2^2 .$$

We obtain the optimal value $\hat{\mathbf{s}}$ by the closed-form solution [SL03]

$$\hat{\mathbf{s}} = (\mathbf{C}^T \mathbf{C})^{-1} \mathbf{C}^T \mathbf{z} .$$

M estimators such as Huber and Cauchy regression can be computed by *iterative reweighted least squares (IRLS)* (Algorithm 4).

Input: $\mathbf{C} \in \mathbb{R}^{m \times n}$, $\mathbf{z} \in \mathbb{R}^m$
Output: estimate $\hat{\mathbf{s}}$

```

1 W =  $\mathbf{I}_m$ 
2 repeat
3    $\mathbf{s}' = (\mathbf{C}^T \mathbf{W} \mathbf{C})^{-1} \mathbf{C}^T \mathbf{W} \mathbf{z}$       ▷ Closed form solution for Equation (2)
4   Update W based on  $\mathbf{s}'$  and  $\rho$  according to Equation (1)
5 until converged
6 return  $\mathbf{s}'$ 
```

Algorithm 4: Iterative Reweighted Least Squares

The main idea of IRLS is to iteratively compute the minimum of a *weighted* ordinary least-squares loss function. The weight of each residual is derived from the respective loss function ρ and the residuals of the previous iteration. The weight function is given by

$$w(r_i) = \begin{cases} \rho'(r_i)/r_i & \text{if } r_i \neq 0 \\ \rho''(r_i) & \text{if } r_i = 0 \end{cases} , \quad (1)$$

where ρ' and ρ'' denote the first and second derivative of the loss function ρ .

To compute the minimum of this weighted residual function, we can arrange the weights into a diagonal matrix $\mathbf{W}$ and obtain a minimiser for the following residual loss function. Let $\mathbf{s}^*$ be the estimate of the previous iteration, then

$$\begin{aligned} L_m(\mathbf{s}') &= \frac{1}{m} \left\| \mathbf{W} (\mathbf{C}^T \mathbf{s}' - \mathbf{z}) \right\|_2^2 \\ &= \frac{1}{m} \sum_{i=1}^m w(r_i(\mathbf{s}^*)) \left((z_i - \mathbf{C}_i^T \mathbf{s}')^2 \right) . \end{aligned} \quad (2)$$

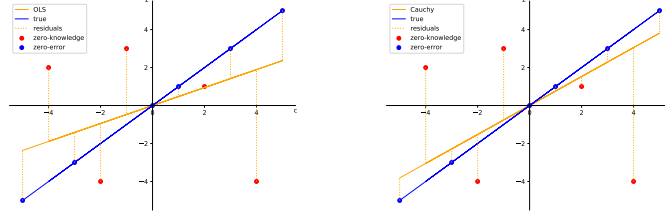


Fig. 5: Residuals of initial OLS estimate (left) and updated residuals after one iteration of Cauchy loss (right).

If the loss function ρ is convex, then the corresponding empirical loss function L_m is also convex and Algorithm 4 converges to the *global* minimum of L_m [HR09] in $\mathcal{O}(n)$ steps [Ars04, BV04]. If $\rho(r)$ is non-convex, but increasing for $r > 0$, Algorithm 4 converges to a *local* optimum in $\mathcal{O}(n)$ steps [HR09]. For a proof, we refer to [MMYSB19]. Each step of the algorithm computes an OLS solution in time $\mathcal{O}(n^2m)$ [HR09].

We illustrate the algorithm based on the example in Figure 3. Figure 5 shows the residuals computed for the initial iteration of OLS (left plot), while the update with the Cauchy loss is shown in the right plot. One can clearly observe that the influence of those points far away from the true line is reduced as their residuals get larger while the regression curve approaches the true solution.

5 Evaluation

In this section, we evaluate the efficacy of Huber and Cauchy regression on synthetic CILWE instances. We generate the CILWE instances in a way that simulates CILWE samples obtained from a side-channel attack against Dilithium. Our results are as follows.

- OLS needs a prohibitive amount of samples even for small concealment rates.
- Huber is comparable to the ILP-based method presented by Ulitzsch et al. [UMTS24], but can solve a slightly larger concealment rate.
- Cauchy required fewer samples and can thus handle a far higher concealment rate than the other methods.

Based on these findings, we later utilise Cauchy regression to launch an end-to-end power side-channel attack on a recently proposed masked Dilithium implementation [CGTZ23] (see Section 6).

We further tried other robust regression algorithms like RANSAC [FB81], however they did not improve upon OLS substantially.

5.1 Simulations

To estimate the efficacy of the presented regression algorithms, we compare them with the ILP-based algorithm presented in prior work, as well as a naive OLS-based algorithm. To this end, we test the algorithms’ performance on synthetic Dilithium CILWE instances, which are generated according to NIST security Level 2, 3, and 5, respectively [BDK+21], and under different concealment rates. For each NIST-level and each concealment rate, we investigate for each regression method introduced in Section 4 how many samples are required to recover the secret key with high probability.

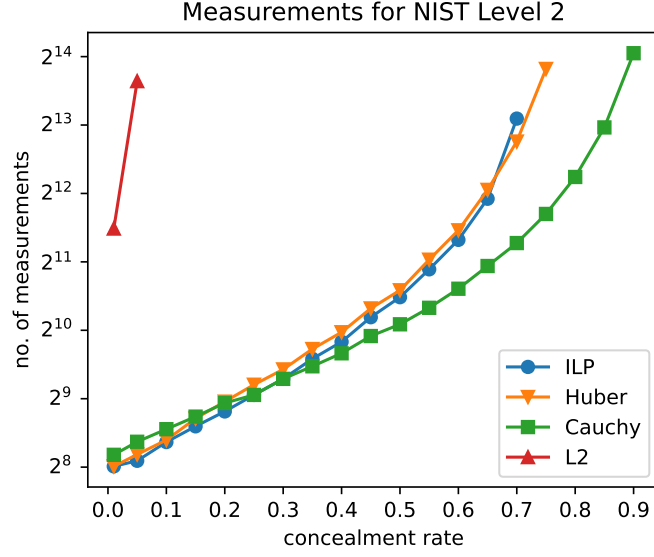
Then, we revisit the experiments of [UMTS24]. To make our observations comparable, we investigate the number of signatures needed to perform our attack (in contrast to just samples). Note that a single signature can give rise to multiple (or no) samples. We find that both Huber and Cauchy regression outperform the results of the original work.

Sample Generation Let m be the number of samples for a CILWE instance. We uniformly sample a secret $\mathbf{s} \in \{-\eta, \dots, \eta\}^n$. Then, we generate m CILWE samples $(z, \mathbf{c})$, where with probability p we generate a zero-knowledge sample and with probability $(1 - p)$ a zero-error sample. For all samples, we ensure $|z| \leq 39$, modelling a priori filtering that side-channel attacks on Dilithium would employ (see Section 3).

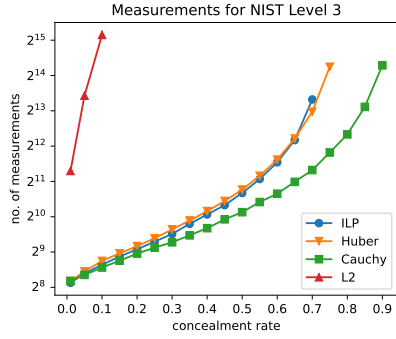
Experimental Setup The entire experiment was steered by a Python-based software, available in the artefact provided with this paper. For comparison, we attempted to solve each instance with ILP, Huber regression, Cauchy regression, and OLS. The first two, we solved with `cvxpy` [DB16, AVDB18] and `Mosek` [ApS24]. For Huber regression, we chose the Huber parameter $\delta = \frac{1}{8}$ as this is both practically and theoretically beneficial for CILWE (see Section 7). Selecting δ involves a trade-off: we require a somewhat small δ for correctness (excluding the zero-knowledge samples), while a somewhat large δ is required for runtime (faster convergence) [HR09]. We experimentally determined the optimal parameter to be at $\frac{1}{8}$, independent of the concealment rate. We solve OLS with `numpy` [HMvdW+20], which we also use in our custom implementation of Algorithm 4 to solve Cauchy regression. We consider Algorithm 4 to have converged if none of the coefficients of $\mathbf{s}$ changed by more than 0.01 for 10 iterations. We ran Levels 2,3 and 5 separately on different VMs (8 cores, 8 GB RAM, Ubuntu 22.04) with a timeout of 120 seconds for each method on each instance and a size limit of 40,000. For each set of parameters, we measured how many CILWE samples were needed to solve 95 out of 100 instances.

Findings Figure 6 summarises our findings. It shows for each NIST-Level and each method how many samples we need, based on the contamination rate, to retrieve at least 95 out of 100 secret keys.

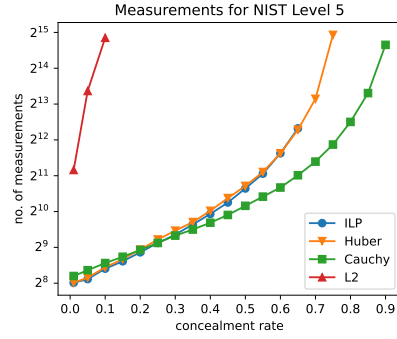
The ILP and Huber regression initially show similar performance. For higher contamination rates, the success rate of the ILP falls behind and then fails altogether. This is due to the growing size and inherent difficulty (NP-hardness)



(a) NIST-2



(b) NIST-3



(c) NIST-5

Fig. 6: These diagrams show how many samples we need for each method based on the concealment rate and the NIST level. We clearly see that ILP and Huber perform similarly (if they succeed) while Cauchy needs significantly fewer samples and thus can handle larger concealment rates. All of these by far outperform OLS.

of the problem. With Cauchy regression, the number of required samples exhibits slower growth. Therefore, it becomes feasible to retrieve the secret key for contamination rates of 90%, which highlights the power of our regression techniques.

Considering the running time of the estimators, we observe quite varying behaviour. For both Huber and Cauchy, the vast majority ($\sim 98\%$) of instances were solved in less than 20 seconds. In contrast, the ILP almost always ran into the timeout. However, in many cases, the solution found up to that point was the correct solution and only its optimality could not be confirmed by the solver. So we still regard these runs as successful.

5.2 Improving UMTS24

We now show that our new methods actually outperform the simulated attack of [UMTS24].

We evaluate the number of signatures required to recover the secret key of Dilithium with NIST security level 2 parameters, under assumed quality metrics of the side-channel and corresponding machine-learning classifier. To do so, we repeat the following experiment by Ulitzsch et al. [UMTS24]: We generate a secret key and a certain number of signatures for that key. We then derive $\ell = 4$ CILWE instances from the generated signatures, assuming an (imperfect) classifier that reveals information about whether $y_{i,j} = 0$. We then filter the samples, discarding all with $|z_{i,j}| > 2\sigma$ and afterwards apply Huber and Cauchy regression.

Ulitzsch et al. considered an attack successful if it was able to recover one out of 20 keys s_1 . Table 2 lists the minimum number of signatures required for an attack strategy to be successful, depending on the quality of the classifier (see Section 2.4). Both Cauchy and Huber regression require fewer signatures. In addition, we can solve additional instances that [UMTS24] was unable to solve, as we can tolerate a much higher concealment rate (see Figure 6a).

TPR	FPR	average p	Huber	Cauchy	[UMTS24]
0.99	0.01	0.22	500,000	400,000	650,000
0.90	0.01	0.24	550,000	500,000	750,000
0.981	0.019	0.30	650,000	550,000	800,000
0.96	0.04	0.48	1,050,000	650,000	-

Table 2: Required signatures in the setting of [UMTS24] on Dilithium NIST security level 2 (best is **bold**). For comparison, we also give the concealment rate.

6 Attacking Masked Dilithium

In this section, we demonstrate an end-to-end proof-of-concept attack on a recently proposed masked Dilithium implementation, configured with first-order masking [CGTZ23]. We demonstrate our attack against all Dilithium NIST security levels. For a given signature, an attacker can use a machine learning-based power side-channel attack to (imperfectly) distinguish whether a specific coefficient $y_{i,j}$ of a masked polynomial lies within the range $\{-\eta \cdot \tau, \dots, -1\}$ or $\{0, \dots, \eta \cdot \tau\}$. This gives rise to multiple ILWE instances from which Cauchy regression can recover the secret key s_1 . In our experiments, ILP and OLS-based methods cannot achieve key recovery.

A recent study [HNP25] targets a similar optimised version of the masked implementation [CGL⁺24]. We confirmed that the targeted optimised implementation exposes a similar side-channel as used in our attack. Therefore, both attacks share a common target and masking order. However, the approaches differ in several key aspects. First, we demonstrate our attack for all NIST security levels, while Hermelink et al. target only NIST security level 2. Second, our machine learning-assisted profiling attack is simpler and requires less extensive profiling (both in terms of collected signatures and computational resources) than Hermelink et al.’s templating attack. Furthermore, our ARM Cortex M4 target device firmware is compiled with the highest level of optimisation (-O3). Prior work has shown that this optimisation level leads to the highest power side-channel resilience for other lattice-based cryptography implementations [SKL⁺20]. However, Hermelink et al.’s attack requires fewer signatures during the attack stage [HNP25].

The rest of this section is structured as follows. We will first describe details about the masked implementation, followed by a leakage assessment of said masked implementation compiled for first-order masking. We root-cause this leakage to micro-architectural leakages in the memory architecture. We then exploit this leakage through a machine learning-assisted profiling attack, demonstrating key recovery through robust regression. For ease of exposition, we will describe our attack using Dilithium NIST security level 2 parameters, if not otherwise noted. Our results do apply to all security levels.

6.1 Describing the Masked Implementation

The targeted masked Dilithium implementation achieves provable security in the t -probing model, which allows an attacker to probe up to t variables through a side-channel [CGTZ23, ISW03]. To protect against a t -probing attacker, the implementation splits each variable into $t+1$ shares. A central component of the targeted implementation is the masked sampling of the coefficients of the polynomials in the error vector $\mathbf{y}$. These coefficients need to be sampled uniformly from the interval $]-\gamma_1, \gamma_1]$, with $\gamma_1 = 2^{17}$ for Dilithium NIST security level 2. This is achieved using the efficient Boolean-to-arithmetic conversion procedure proposed by Coron et al. [CGTZ23].

The implementation samples each coefficient $y_{i,j}$ as follows. First, it generates the Boolean shares used to mask $y_{i,j}$. To do so, it samples n_{shares} (where $n_{shares} - 1$ is the *masking order*) uniformly random 32-bit *unsigned* integers. The implementation then interprets the μ least significant bits of each sampled integer as one of the coefficient $y_{i,j}$'s Boolean shares. At this point, the Boolean shares mask an unsigned integer in the range $[0, 2^\mu[$ where $\mu = 1 + \log \gamma_1$. This intermediate value, denoted by $y_{i,j}^{tmp}$ determines the value of the sampled coefficient $y_{i,j}$, a fact that we exploit in our attack. We highlight that this $y_{i,j}^{tmp}$ is *never explicitly stored or unmasked in the implementation*.

The Boolean-to-arithmetic conversion procedure then transforms the Boolean-masked shares into an arithmetic masking modulo q , yielding a representation that supports masked arithmetic operations. The conversion operates in three steps:

1. Conversion from a μ -bit Boolean masking to an arithmetic masking modulo 2^k , for a parameter $k \geq \mu$, using the method of [BCZ18]. This step is an approximate conversion and introduces small conversion errors. It is performed primarily within the `impcnvBA_rec64()` function.
2. Application of a *modulus switching* technique to convert from arithmetic masking modulo 2^k to modulo q . This step is done by calling the function `exact_modulus_switching()`.
3. Finally, the masked coefficient's range is adjusted by subtracting $\gamma_1 - 1 = 2^{\mu-1} - 1$ from the first share, achieving the required coefficient range of $]-\gamma_1, \gamma_1]$.

The described methods' complexity is independent of the bit size μ and the size of the modulus q [CGTZ23], an advantage over various Boolean-to-arithmetic conversion procedures that have been proposed by prior work [SPOG19, CGMZ22, BC22]. A further optimised version of this conversion procedure reduces the mask bit size and uses an approximation technique instead of modulus switching to further increase efficiency [CGL⁺24].

6.2 Identifying A Micro-Architectural Leakage In The Conversion Function

Our attack specifically targets the first step of this conversion. We demonstrate that this Boolean-to-arithmetic conversion suffers from micro-architectural leakage when executed on an ARM Cortex M4. This leakage can be exploited to attack a first-order masked Dilithium implementation in all NIST security level configurations. In the following, we describe our experimental setup and then explain this leakage in detail.

Experimental Setup Throughout this section, all measurements are performed using two identical ChipWhisperer Lite [Inc18] devices with CW308 UFO target boards and STM32F415RGT6 ARM Cortex M4 targets. One of these devices will act as the *profiling device*, used for leakage analysis and profiling. The

other device will be the *victim device*, used for mounting the attack. Both target boards run on a 7,372,800 Hz clock. We collect traces through an ADC with 4 samples/cycle and 10-bit resolution. We compile the target firmware using arm-none-eabi-gcc 13.2.1 with optimisation level 3 (-O3).

Due to memory constraints, the ChipWhisperer cannot trace the full resource-heavy masked signature generation. We therefore precompute the masked signatures on an x86 Ubuntu 20.04 server machine. During this precomputation, we gather internal inputs to the function under investigation for later computation and tracing on the target devices.

Power Side-Channel Leakage We identify a leaking point within the generation function `gen_y()` for the coefficients of $\mathbf{y}$. More specifically, it is in the Boolean-to-arithmetic conversion function `impcnvBA_rec64()` used for the masked sampling of coefficients. Listing 1 shows the relevant code of this function for the first-order masked version of Dilithium. For first-order masking (two shares, cf. Listing 1, Line 6) of Dilithium NIST level 2, the implementation generates two uniformly random 32-bit unsigned integers. The 18 least significant bits are used as the Boolean shares masking the $\mathbf{y}_{i,j}$ coefficient. As explained in Section 6.1, if the Boolean shares mask an intermediate value $\mathbf{y}_{i,j}^{tmp}$ of $2^{17} = 131072$, the corresponding coefficient $\mathbf{y}_{i,j}$ will be 0.

The conversion function itself samples two additional 64-bit integers `r1` and `r2` to prevent unmasking of the coefficient during the conversion (Listing 1, Line 8,9). While the *t-probing model* demonstrates security at the C code level, as shown in [CGTZ23], it is important to note that this proof does not account for micro-architectural leakage.

```

1 // *D_ - Pointer to array of arithmetic shares [output]
2 // *x - Pointer to array of Boolean shares [input]
3 // n - Number of shares (n-1 = masking order) [input]
4 static void impcnvBA_rec64(uint64_t *D_, uint64_t *x, int n) {
5     if (n==2) {
6         uint64_t r1=rand64();
7         uint64_t r2=rand64();
8         uint64_t y0=(x[0] ^ r1) ^ r2;
9         uint64_t y1=x[1] ^ r1;
10        uint64_t y2=x[2] ^ r2; // x[2] = 0
11
12        uint64_t z0=y0 ^ Psi64(y0,y1);
13        uint64_t z1=Psi64(y0,y2);
14
15        D_[0]=y1 ^ y2;
16        D_[1]=z0 ^ z1;
17
18        return;
19    }
20    [...]
21 }

```

Listing 1: First-order masked ($n_{shares} = 2$, cf. Line 6) randomness vector $\mathbf{y}$ coefficient generation through masked Boolean-to-arithmetic share conversion.

Using Welch’s t-test, we show that the distribution of power consumption during the execution of the Boolean-to-arithmetic conversion function, for shares masking a zero (fixed) versus non-zero (random) coefficient of $\mathbf{y}$, is distinguishable. Note that this is a *first-order leakage* scenario, as the attacker aims to gain knowledge about the secret masked coefficient $\mathbf{y}_{i,j}$ itself, instead of probing the individual shares. We execute the conversion function on the target device. In more detail, we collect 25,000 traces for the conversion of random

Boolean shares resulting in random coefficient values and further 25,000 traces for random Boolean shares resulting in zero coefficients $\mathbf{y}_{i,j}$ after the Boolean-to-arithmetic conversion. Recall that the Boolean shares result in $\mathbf{y}_{i,j} = 0$ if they mask the intermediate value $\mathbf{y}_{i,j}^{tmp} = 131072$ for NIST security level 2. Figure 7a depicts the univariate t-values. The plot shows peaks crossing a *t-value* boundary of ± 4 , indicating significant information leakage with p-value < 0.001 .

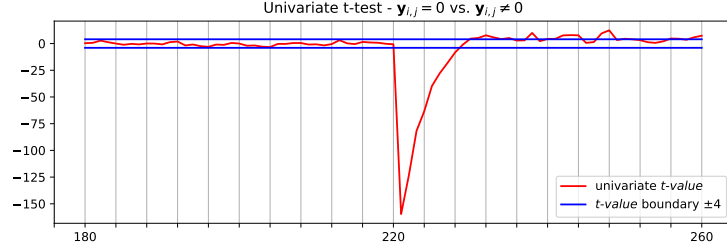
Micro-Architectural Leakage We align the power traces with the assembly instructions to identify and classify the leakage. As shown in Figure 7b, the leaking instruction is `ldr r4, [r1, #8]` (offset 0x24), loading the second Boolean share `x[1]` from memory. Since no memory operation including a statistically unrelated value is executed between this instruction and the previous load of the first Boolean share `x[0]` in `ldr r5, [r1, #0]` (offset 0x00), we conjecture that the *Hamming Distance* (HD) between the loaded shares is leaked when the second share overwrites the first share in an assumed internal memory register. The leaked HD is equal to the *Hamming Weight* (HW) of the result of an XOR operation on the two shares, since the bits are only set or reset, thus increasing or decreasing power consumption, if they differ between shares. As in the targeted Boolean share representation XOR is the *unmasking operation*, the leaked HW is that of the intermediate value $\mathbf{y}_{i,j}^{tmp}$. We highlight that $\mathbf{y}_{i,j}^{tmp}$ determines $\mathbf{y}_{i,j}$ – as such, leaking information about the HW of $\mathbf{y}_{i,j}^{tmp}$ also leaks information about the HW of $\mathbf{y}_{i,j}$.

A similar leakage was first described in [PV17], formally classified and its root-cause assumed in [MPW21] for the target STM32F415RGT6 Cortex M4 and other microcontrollers. The authors define this as a *load-adjacent sequential inter-instruction micro-architectural power leakage* due to the *Memory Remnant Effect*. The corresponding C expressions are shown in Listing 1 Line 10 and 11, where consecutively the shares are loaded and XOR’ed with the random mask(s).

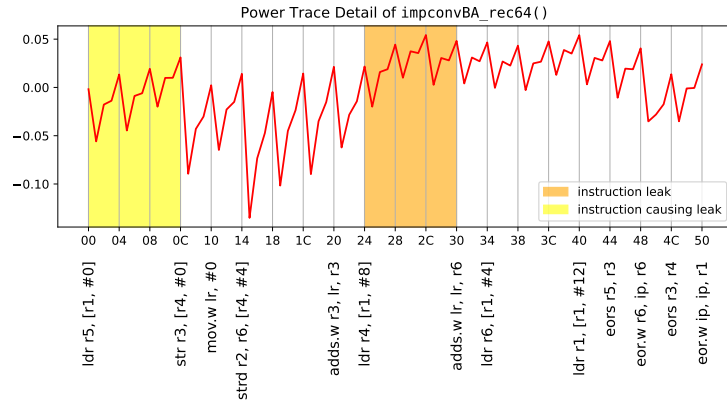
6.3 End-To-End Attack On Masked Dilithium

We now describe an end-to-end *machine learning-based profiling attack* on first-order masked Dilithium (NIST security level 2 parameter set). Our attack consists of two stages, the *profiling stage* and the *attack stage*. The attack proceeds as follows.

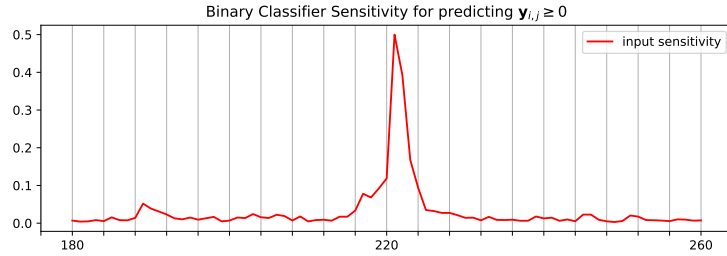
In the *profiling stage*, the attacker collects power traces from a device completely under their control (the profiling device). The attacker uses this to measure power consumption during the signing process. The attacker conducts these measurements with a known secret key (and thus, also known $\mathbf{y}$ for each signature). The attacker then trains a machine learning classifier to reveal information about $\mathbf{y}$ ’s coefficients, using power traces collected during the execution of the `impcnvBA_rec64()` conversion function. For our experiments, we will train the classifier to distinguish between the cases $\mathbf{y}_{i,j} < 0$ and $\mathbf{y}_{i,j} \geq 0$ in a $\mathbf{z}$ range chosen for high probability of zero-error $\mathbf{y}_{i,j} = 0$ samples. For a coefficient $\mathbf{y}_{i,j}$ to be zero-error, a priori, the corresponding $\mathbf{z}_{i,j}$ must lie within the range $[-\eta \cdot \tau, +\eta \cdot \tau]$ with $\eta \cdot \tau = 78$ for NIST security level 2 (see Section 3).



(a) Univariate fixed vs. random t-test for the groups $y_{i,j} = 0$ and $y_{i,j} \neq 0$ and (blue) t-value boundary of ± 4 (p-value < 0.001). Each group contains 25,000 traces.



(b) Reference power trace aligned to the function's assembly instructions when compiled with arm-none-eabi-gcc 13.2.1 optimization level 3 (-O3). The identified leaking instruction and the instruction causing the leakage are highlighted in orange and yellow respectively.



(c) Input sensitivity analysis trace of the profiling stage binary classifier trained to predict coefficients $y_{i,j} \geq 0$.

Fig. 7: These diagrams show analysis details of the `imconvBA_rec64()` conversion function's power traces. It is visible that the t-test indicated leakage (a) linked to the `ldr r4, [r1, #8]` instruction (offset 0x24) in (b) align with the peak in the SA (c), indicating a first-order leakage scenario. The *full* analysis plots are shown in Appendix A, Figure 9.

In the *attack stage*, we assume that the attacker can trigger the generation of signatures on a victim device and collect power traces of the generation of these signatures. We assume that all signatures are generated using the same secret key and that the attacker can observe the Dilithium signature values $(\mathbf{z}, c)$, where $\mathbf{z} \in R_q^\ell, c \in R_q$. Typically, the victim device is similar to the profiling device, e.g., follows the same circuit schematic and uses the same standard cell library. For each generated signature, the attacker collects the power traces corresponding to the generation of the masked $\mathbf{y}$ vector. The attacker then uses these power traces as an input to the classifier trained in the profiling stage to predict whether coefficients of $\mathbf{y}$ are greater than or equal to zero. If the classifier predicts that $\mathbf{y}_{i,j} \geq 0$, the attacker derives an ILWE sample from the corresponding signature values. This ILWE sample is given by $(\mathbf{z}_{i,j}, \mathbf{c})$, where $\mathbf{z}_{i,j} \in \mathbb{Z}$, and $\mathbf{c} \in \mathbb{Z}^n$ is derived from the challenge polynomial c . To recover the secret key, the attacker collects these samples into ℓ ILWE instances. The attacker then recovers the partial secret key $\mathbf{s}_1$ from these instances using *Cauchy* regression. Having obtained $\mathbf{s}_1$, the attacker can perform universal forgery [GBP18, RJH⁺18]. Next, we will describe our experimental results.

Profiling Stage In the profiling stage, we collect 64,000 traces of the `impcnvBA_rec64()` conversion function’s execution with 400 data points each using the *profiling device* for chosen Boolean share inputs. We then train a *Convolutional Neural Network*-based binary classifier. It is trained to predict whether a collected power trace corresponds to the conversion of Boolean shares masking a small independent error $\mathbf{y}_{i,j} \geq 0$ coefficient, in a coefficient target range with increased likelihood of $\mathbf{y}_{i,j} = 0$ (zero-error samples) as explained in this section’s introduction. We choose the inputs to the traced conversion function to be pairs of random Boolean shares masking $\mathbf{y}$ coefficients in the range $\mathbf{y}_{i,j} \in [-16, +16]$, to build a representative training subset in regards to HWs, which also lies within our optimized coefficient target range (cf. Fig. 8). Our training set consists of 2000 traces per profiling range $\mathbf{y}$ coefficient value (i.e., $32 \cdot 2000 = 64,000$ traces in total).

Classifier Analysis The classifier’s layout and training characteristics are shown in Table 3 and Table 4, respectively. We calculate all metrics for a *prediction threshold* of 80%, which means that classifications are only accepted if the classifier indicates a probability greater than the selected threshold. The threshold is chosen to balance FPR and Recall. The higher TPR on the attack data can be explained by differences in the distribution of the $\mathbf{y}$ coefficients in the training vs. the attack data set.

We did not achieve training a classifier predicting $\mathbf{y}_{i,j} = 0$ with sufficient precision for the targeted masked implementation. Trained classifiers were not able to differentiate between the target HW of the intermediate value $\mathbf{y}_{i,j}^{tmp}$ (with $\text{HW}(131072) = 1$), and other small HWs of $\mathbf{y}_{i,j}^{tmp}$. We attribute this to noise in the target device’s power supply as well as the limited measurement resolution.

However, due to the bit-representation structure of coefficients in the target range depicted in Figure 8, classifying if a coefficient is smaller or greater-equal zero is possible with high accuracy and precision (cf. Table 4).

Analysis of the classifiers’ training input *Sensitivity Analysis* (SA) trace shows that it learns mostly from the previously identified first-order HW leakage, as the alignment of t-test and assembly instructions with the classifier’s input sensitivity plot depicted in Figure 7b and 7c shows. SA is a technique that leverages the input activation gradients of a classifier and can locate exactly all points of interest in the training sequence (power trace), by quantifying their contribution to the learnt *leakage function*. Specifically, the SA trace represents the partial derivatives of every output coordinate of the neural network with respect to the network inputs, thus characterising the effect of a slight change in each point of input on the classification outcome.

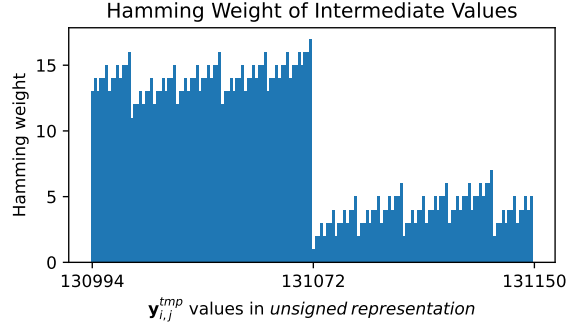


Fig. 8: This figure shows the Hamming weights of the intermediate values $y_{i,j}^{tmp}$ corresponding to $y_{i,j}$ coefficients in the range $[-78, 78]$. Note that $y_{i,j}^{tmp} = 131072$ corresponds to $y_{i,j} = 0$. One can clearly distinguish between $y_{i,j} \geq 0$, for which $y_{i,j}^{tmp} \geq 131072$, and $y_{i,j} < 0$, for which $y_{i,j}^{tmp} < 131072$.

Layer (type)	Output Shape	Param #
Conv1D	(None, 393, 64)	576
MaxPooling1D	(None, 196, 64)	0
Flatten	(None, 12,544)	0
Dense	(None, 1)	12,545
Total params		39,365
Trainable params		13,121
Non-trainable params		0

Table 3: Profiling binary classifier model summary.

Metric	Training	Attack
Accuracy	90.77	89.76
Precision	94.30	87.75
TPR	86.79	92.59
FPR	5.25	13.13

Table 4: Binary classifier profiling (training) and attack characteristics (in %) for a prediction threshold of 80%. Precision is the fraction of samples correctly classified as $\mathbf{y}_{i,j} \geq 0$ (out of all samples classified as $\mathbf{y}_{i,j} \geq 0$) (see Section 2.4).

Attack Stage We perform the computation-heavy generation of masked signatures on a server. In addition to the public components of each signature $\mathbf{z}_{i,j}$ and c_i , we collect the two internally generated Boolean shares that represent the respective coefficients $\mathbf{y}_{i,j}$. We also employ the filtering optimisation explained in Section 3, dismissing all samples for which the absolute value of the corresponding $|\mathbf{z}_{i,j}|$ is above a certain threshold. For each security level, we empirically determine a filtering threshold that minimises the number of required signatures. In doing so, we balance out zero-error prevalence with the probability of obtaining a usable sample.

In the next step, we execute the `impcnvBA_rec64()` conversion function on the victim device with the gathered Boolean share pairs as input and use the profiling stage classifier to predict whether a recorded trace is likely to show the conversion of Boolean-to-arithmetic shares that mask a $\mathbf{y}_{i,j} \geq 0$ coefficient. For each trace predicted to show such a computation, we derive the corresponding ILWE sample from the values $\mathbf{z}_{i,j}$ and c . Finally, we solve the ℓ constructed ILWE instances using the Python script from Section 5.1 to recover the secret key $\mathbf{s}_1$.

Evaluation We successfully execute the described attack against first-order masked Dilithium configured for the NIST security levels 2, 3, and 5.

Table 5 depicts, for each security level, the number of required signatures, the number of derived ILWE samples, as well as the percentage of zero-knowledge samples ($\mathbf{y}_{i,j} < 0$), small, independent error samples ($\mathbf{y}_{i,j} > 0$) and zero-error samples ($\mathbf{y}_{i,j} = 0$) along with the runtime of the Cauchy regression-based algorithm used to recover $\mathbf{s}_1$.

7 Sample Bound for Huber Regression on Dilithium

In this section, we show that Huber regression is indeed a sound and efficient estimator for the CILWE samples arising from Dilithium in the setting of [UMTS24]. Note however, that our proof can be generalised in two ways. First, Dilithium samples have a very distinct form as $\mathbf{c}$ is highly structured. One can relax assumptions on this and for example assume that is some subgaussian distribution

NIST Security Level	2	3	5
Required Signatures	590,000	2,550,000	2,620,000
ILWE Samples	122,000	275,000	360,000
$ \mathbf{z}_{i,j} $ -Filter Threshold	50	100	80
Zero-Knowledge [%]	12.25	15.10	14.45
Small, Independent Error [%]	85.97	83.77	84.34
Zero-Error [%]	1.78	1.13	1.21
Cauchy Regression Algorithm [s]	21	100	78

Table 5: Experimental results for the attack on first-order masked Dilithium under all NIST security levels.

as introduced by [BDE⁺18]. Secondly, we assume CILWE samples which are either zero-knowledge or zero-error samples. However, we can also tolerate small independent errors instead of strictly zero-error samples, if we choose constants and parameters accordingly (see [TJSO14]).

We specialise Definition 3.2 to a more explicit statement.

Definition 7.1 (CILWE for Dilithium).

Let $\mathbf{s} \in \{-\eta, \dots, +\eta\}^n$ be the secret key. Let $\chi_{\mathbf{c}}$ and $\chi_{\mathbf{y}}$ be two probability distributions. $\chi_{\mathbf{c}}$ returns a sample from $\{-1, 0, 1\}^n$ which consists of exactly τ non-zero entries with equal probabilities. Let the error $\mathbf{y}_i$ be a sample from $\chi_{\mathbf{y}}$. Find $\mathbf{s}$ given m samples $(\mathbf{z}_i, \mathbf{C}_i)$ for $i = \{1, \dots, m\}$, where each row $\mathbf{C}_i \leftarrow_r \chi_{\mathbf{c}}$ independently and

$$\mathbf{z}_i = \mathbf{C}_i^T \mathbf{s} + \mathbf{y}_i.$$

Let $\chi_{\mathbf{y}}$ be a concealed distribution with known concealment rate p such that each sample $\mathbf{y}_i$ follows a mixture distribution

$$\mathbf{y}_i \sim (1 - p) \delta_0 + p \chi_{\tilde{\mathbf{y}}},$$

where δ_0 is the point mass distribution at 0 and $\chi_{\tilde{\mathbf{y}}}$ draws samples from $[-\gamma_1, \gamma_1]$ uniformly at random, then computes $\mathbf{z}_i = \mathbf{C}_i^T \mathbf{s} + \mathbf{y}_i$ and repeats if $|\mathbf{z}_i| > 2\sigma$.

One can easily observe that the error distribution matches the samples we obtain from the side channel simulation of [UMTS24]. Either we obtain error-free samples (samples with an error from a point mass distribution at zero) or zero-knowledge samples from the normal generation process of Dilithium. However, we adapted the threshold for rejection sampling to be much closer to zero in order to have less false positives (see Section 3).

We are now ready to state our main theorem, that states that Huber regression is able to recover the secret $\mathbf{s}$ from polynomially many CILWE samples (polynomially in n and $\frac{1}{1-p}$).

Theorem 7.2 (Sample Complexity of Huber Regression for CILWE on Dilithium). Let $(\mathbf{z}_i, \mathbf{C}_i)$ for $i \in \{1, \dots, m\}$ be CILWE samples with concealment rate p according to Definition 7.1. Algorithm 3 with Huber loss (Definition 4.2) with $\delta > 0$ recovers the secret $\mathbf{s}$ in polynomial time given

$$m \geq \max \left\{ \frac{2^{12}n^2\delta^2}{(1-p)^2\tau}, \frac{2^{10}n^2}{(1-p)^2} \right\}$$

with a probability of at least

$$1 - \left(n \exp \left(-\frac{(1-p)m}{8n} \right) + \frac{1}{8} \right).$$

The proof proceeds similar to [TJSO14]. We first show that sufficiently many *informative samples* fall into the quadratic region of the Huber loss. The loss is strongly convex in this region, such that we can obtain a norm bound on the distance of the true secret $\mathbf{s}$ to the estimate $\hat{\mathbf{s}}$, i.e. $\|\mathbf{s} - \hat{\mathbf{s}}\|_2$ which depends on the number of samples m .

Let us first define the set of *informative samples* A (those zero-error samples with a small enough norm)

$$A = \{i \mid \mathbf{y}_i = 0, \|\mathbf{C}_i\|_2 \leq \alpha\sqrt{m}\}.$$

Here, we choose the radius r and the norm requirement α as

$$r = \frac{2^5 n \delta}{(1-p)\sqrt{\tau}\sqrt{m}} \quad \alpha = \frac{\sqrt{\tau}(1-p)}{2^5 n}. \quad (3)$$

With these choices⁵ we have that for all regression estimates that are close enough to the true unknown secret $\mathbf{s}$, i.e., for all $\mathbf{s}' \in \mathcal{B}_r(\mathbf{s})$ (with r as above), it holds

$$|\mathbf{z}_i - \mathbf{C}_i^T \mathbf{s}'| = |\mathbf{C}_i^T (\mathbf{s}' - \mathbf{s})| \leq \underbrace{\|\mathbf{C}_i\|_2}_{\leq \alpha\sqrt{m}} \underbrace{\|(\mathbf{s}' - \mathbf{s})\|_2}_{\leq r} = \delta,$$

implying that all samples A fall into the quadratic region of the Huber loss function. Following [TJSO14], we need to show two requirements for the convergence proof:

First, we need to lower bound the smallest eigenvalue of $\mathbf{C}_A^T \mathbf{C}_A$ (the *Gram matrix* constructed from the informative samples). This is essential for all regression techniques as the Gram matrix needs to be invertible (i.e., all eigenvalues must be larger than zero). It also resembles the second order gradient of the loss function, which is why we are interested in its precise value.

Lemma 7.3 (Smallest Eigenvalue of $\mathbf{C}_A^T \mathbf{C}_A$ for Dilithium). *Consider $A = \{i \mid \mathbf{y}_i = 0, \|\mathbf{C}_i\|_2 \leq \alpha\sqrt{m}\}$, the index set of error-free samples. Then, whenever $m \geq \frac{2^{10}n^2}{(1-p)^2}$ it holds with probability at least $1 - n \exp \left(-\frac{(1-p)m}{8n} \right)$ that the smallest eigenvalue of $\frac{1}{m} \mathbf{C}_A^T \mathbf{C}_A$ satisfies*

$$\lambda_{\min} \left(\frac{1}{m} \mathbf{C}_A^T \mathbf{C}_A \right) \geq \frac{(1-p)\tau}{2n}.$$

⁵ We will see that the radius r is chosen to comply with Equation (6) in the proof.

Proof. The proof follows similar steps as [DKM⁺25, Lemma 5.1.] and can be found in Appendix B.

Secondly, we need to bound the first order gradient of the loss function at the true value $\mathbf{s}$.

Lemma 7.4 (Gradient of the Loss Function for Dilithium). *The norm of the gradient of the empirical Huber loss at the true value of $\mathbf{s}$ is bounded with probability $1 - \frac{1}{8}$*

$$\|\nabla L_m(\mathbf{s})\|_2 \leq \frac{8\delta\sqrt{\tau}}{\sqrt{m}}.$$

Proof. See Appendix C.

Given these two lemmas, we can now prove Theorem 7.2.

Proof (of Theorem 7.2). We start by decomposing the loss $L_m(\mathbf{s}')$ for any $\mathbf{s}' \in \mathbb{R}^n$, into the loss caused by error-free samples ($i \in A$) and zero-knowledge samples ($i \in \bar{A}$):

$$L_m(\mathbf{s}') = \underbrace{\frac{1}{m} \sum_{i \in A} \rho_\delta(\mathbf{z}_i - \mathbf{C}_i^T \mathbf{s}')}_{L_n^A(\mathbf{s}')} + \underbrace{\frac{1}{m} \sum_{i \notin A} \rho_\delta(\mathbf{z}_i - \mathbf{C}_i^T \mathbf{s}')}_{L_n^{\bar{A}}(\mathbf{s}')}.$$

As the Huber loss is convex, it holds for the concealed loss part, $L_m^{\bar{A}}$ and all $\mathbf{s}'$

$$L_m^{\bar{A}}(\mathbf{s}') \geq L_m^{\bar{A}}(\mathbf{s}) + \nabla L_n^{\bar{A}}(\mathbf{s})^T (\mathbf{s}' - \mathbf{s}). \quad (4)$$

Furthermore, the loss on the error-free samples, L_m^A , is strongly convex in a neighbourhood around the true $\mathbf{s}$, i.e. $\mathcal{B}_r(\mathbf{s}) := \{\mathbf{s}' \mid \|\mathbf{s}' - \mathbf{s}\|_2 \leq r\}$ with r as defined in Equation (3). This strong convexity implies $\forall \mathbf{s}' \in \mathcal{B}_r(\mathbf{s})$ (see, e.g., [TJSO14, BV04])

$$L_m^A(\mathbf{s}') \geq L_m^A(\mathbf{s}) + \nabla L_m^A(\mathbf{s})^T (\mathbf{s}' - \mathbf{s}) + \frac{\lambda_{\min}}{2} \|\mathbf{s}' - \mathbf{s}\|_2^2$$

where $\lambda_{\min}$ is the smallest eigenvalue of the Hessian of $\nabla_{\mathbf{s}'}^2 L_n^A(\mathbf{s}') = \frac{1}{m} \mathbf{C}_A^T \mathbf{C}_A$. By Lemma 7.3 we have $\lambda_{\min} \left(\frac{1}{m} \mathbf{C}_A^T \mathbf{C}_A \right) \geq \frac{(1-p)\tau}{2n}$, therefore

$$L_m^A(\mathbf{s}') \geq L_m^A(\mathbf{s}) + \nabla L_m^A(\mathbf{s})^T (\mathbf{s}' - \mathbf{s}) + \frac{(1-p)\tau}{4n} \|\mathbf{s}' - \mathbf{s}\|_2^2. \quad (5)$$

Let's now recombine Equations (4) and (5). We have that for all $\mathbf{s}'$, $L_m^A(\mathbf{s}') + L_m^{\bar{A}}(\mathbf{s}') = L_m(\mathbf{s}')$. By the linearity of the gradient we get

$$L_m(\mathbf{s}') \geq L_m(\mathbf{s}) + \nabla L_m(\mathbf{s})^T (\mathbf{s}' - \mathbf{s}) + \frac{(1-p)\tau}{4n} \|\mathbf{s}' - \mathbf{s}\|_2^2.$$

Let $\hat{\mathbf{s}}$ denote the minimiser of the *empirical loss* L_m , i.e., $L_m(\hat{\mathbf{s}}) \leq L_m(\mathbf{s})$. As the equation above holds for all $\mathbf{s}' \in \mathcal{B}_r(\mathbf{s})$, in particular also for $\hat{\mathbf{s}} \in \mathcal{B}_r(\mathbf{s})$, we have

$$\begin{aligned} L_m(\hat{\mathbf{s}}) &\geq L_m(\mathbf{s}) + \nabla L_m(\mathbf{s})^T(\hat{\mathbf{s}} - \mathbf{s}) + \frac{(1-p)\tau}{4n} \|\hat{\mathbf{s}} - \mathbf{s}\|_2^2 \\ &\geq L_m(\hat{\mathbf{s}}) + \nabla L_m(\mathbf{s})^T(\hat{\mathbf{s}} - \mathbf{s}) + \frac{(1-p)\tau}{4n} \|\hat{\mathbf{s}} - \mathbf{s}\|_2^2. \end{aligned}$$

We now subtract $L_m(\hat{\mathbf{s}})$ and with help of Cauchy-Schwarz we get

$$\begin{aligned} \|\hat{\mathbf{s}} - \mathbf{s}\|_2^2 &\leq \frac{4n}{(1-p)\tau} \nabla L_m(\mathbf{s})^T(\hat{\mathbf{s}} - \mathbf{s}) \\ &\leq \frac{4n}{(1-p)\tau} \|\nabla L_m(\mathbf{s})\|_2 \|\hat{\mathbf{s}} - \mathbf{s}\|_2. \end{aligned}$$

We invoke Lemma 7.4 and get

$$\|\hat{\mathbf{s}} - \mathbf{s}\|_2 \leq \frac{4n}{(1-p)\tau} \frac{8\delta\sqrt{\tau}}{\sqrt{m}} = \frac{2^5\delta n}{(1-p)\sqrt{\tau}\sqrt{m}} \leq r. \quad (6)$$

In Algorithm 3, we round $\hat{\mathbf{s}}$. Since we want to round to the true solution, we need that $\|\hat{\mathbf{s}} - \mathbf{s}\|_\infty < \frac{1}{2}$. We enforce this by requiring $\|\hat{\mathbf{s}} - \mathbf{s}\|_2 < \frac{1}{2}$. Rearranging for m gives the statement. The success probability is simply the probability that the events in Lemmas 7.3 and 7.4 occur jointly. $\square$

8 Conclusion

This paper introduces the Concealed ILWE problem, a framework that naturally models side-channel and fault injection attacks on Fiat–Shamir with Aborts signature schemes. We introduce two new regression methods, Huber and Cauchy regression, to cryptanalysis. We show that instances of Concealed ILWE can be solved in polynomial time with polynomially many samples using Huber regression. Furthermore, we demonstrate that the problem can be solved efficiently with Cauchy regression in practice. Our simulations show that OLS is insufficient to tackle CILWE, ILP and Huber lead to similar results, and Cauchy regression outperforms ILP-based approaches. Cauchy’s efficacy enables a novel, simple, machine learning-based power side-channel attack on a recently proposed masked Dilithium implementation, for which ILP and OLS cannot achieve key recovery.

Acknowledgments

We thank the reviewers for their valuable input. This work is funded by the European Union’s Horizon Europe framework Programme (HORIZON) under the ERA Chair scheme with grant agreement no. 101087126. This work is funded by

the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy – EXC 2092 CASA – 390781972. This work is supported with funds from the Ministry of Science, Research and Culture of the State of Brandenburg within the Centre for Quantum Technologies and Applications (CQTA). This work is also funded by the Federal Ministry of Education and Research of Germany (BMBF) through the program EASEPROFIT, project identification number: 16KIS2125, the program PQ-CCA, project identification number: 16KIS2109, and the program “Souverän. Digital. Vernetzt.” Joint project 6G-RIC, project identification number: 16KISK030.

GEFÖRDERT VOM



**Bundesministerium
für Bildung
und Forschung**

Bibliography

- [ApS24] MOSEK ApS. *MOSEK Optimizer API for Python 11.0.7*, 2024.
- [Ars04] Olcay Arslan. Convergence behavior of an iterative reweighting algorithm to compute multivariate m-estimates for location and scatter. *Journal of Statistical Planning and Inference*, 118(1):115–128, 2004.
- [AVDB18] Akshay Agrawal, Robin Verschueren, Steven Diamond, and Stephen Boyd. A rewriting system for convex optimization problems. *Journal of Control and Decision*, 5(1):42–60, 2018.
- [BBE⁺19] Gilles Barthe, Sonia Belaïd, Thomas Espitau, Pierre-Alain Fouque, Mélissa Rossi, and Mehdi Tibouchi. Galactics: Gaussian sampling for lattice-based constant-time implementation of cryptographic signatures, revisited. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2147–2164, 2019.
- [BC22] Olivier Bronchain and Gaëtan Cassiers. Bitslicing arithmetic/-boolean masking conversions for fun and profit: with application to lattice-based kems. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(4):553–588, Aug. 2022.
- [BCZ18] Luk Bettale, Jean-Sébastien Coron, and Rina Zeitoun. Improved high-order conversion from boolean to arithmetic masking. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(2):22–45, May 2018.
- [BDE⁺18] Jonathan Bootle, Claire Delaplace, Thomas Espitau, Pierre-Alain Fouque, and Mehdi Tibouchi. LWE without modular reduction and improved side-channel attacks against BLISS. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part I*, volume 11272 of *Lecture Notes in Computer Science*, pages 494–524, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Cham, Switzerland.
- [BDK⁺] Shi Bai, Leo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehle. Dilithium reference implementation. <https://github.com/pq-crystals/dilithium>. [Online; accessed 22-Jun-2023].
- [BDK⁺21] Shi Bai, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-dilithium, algorithm specifications and supporting documentation, 2021. <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf>.
- [BV04] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [BVC⁺23] Alexandre Berzati, Andersson Calle Viera, Maya Chartouny, Steven Madec, Damien Vergnaud, and David Vigilant. Exploit-

- ing intermediate value leakage in dilithium: A template-based approach. Cryptology ePrint Archive, Paper 2023/050, 2023.
- [CGL⁺24] Jean-Sébastien Coron, François Gérard, Tancrede Lepoint, Matthias Trannoy, and Rina Zeitoun. Improved high-order masked generation of masking vector and rejection sampling in dilithium. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(4):335–354, Sep. 2024.
 - [CGMZ22] Jean-Sébastien Coron, François Gérard, Simon Montoya, and Rina Zeitoun. High-order table-based conversion algorithms and masking lattice-based encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(2):1–40, Feb. 2022.
 - [CGTZ23] Jean-Sébastien Coron, François Gérard, Matthias Trannoy, and Rina Zeitoun. Improved gadgets for the high-order masking of dilithium. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):110–145, Aug. 2023.
 - [DB16] Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.
 - [DDLL13] Léo Ducas, Alain Durmus, Tancrede Lepoint, and Vadim Lyubashevsky. Lattice signatures and bimodal gaussians. In *Annual Cryptology Conference*, pages 40–56. Springer, 2013.
 - [DKM⁺25] Simon Damm, Nicolai Krauss, Alexander May, Julian Nowakowski, and Jonas Thietke. One bit to rule them all – imperfect randomness harms lattice signatures. to appear in: PKC and eprint, 2025.
 - [FB81] Martin A Fischler and Robert C Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
 - [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Proceedings on Advances in Cryptology—CRYPTO ’86*, page 186–194, Berlin, Heidelberg, 1987. Springer-Verlag.
 - [GBP18] Leon Groot Bruinderink and Peter Pessl. Differential fault attacks on deterministic lattice signatures. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(3):21–43, Aug. 2018.
 - [HMvdW⁺20] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke,

- and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.
- [HNP25] Julius Hermelink, Kai-Chun Ning, and Richard Petri. Finding and protecting the weakest link: On side-channel attacks on masked ML-DSA. *IACR Cryptol. ePrint Arch.*, page 276, 2025.
- [HR09] Peter J Huber and Elvezio M Ronchetti. *Robust statistics, second edition*. Wiley series in probability and statistics. John Wiley & Sons, Inc, Hoboken, N.J, 2nd ed. edition, 2009.
- [Hub64] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.
- [Inc18] NewAE Technology Inc. *CW1173: ChipWhisperer-Lite*, 2018.
- [ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In *Advances in Cryptology-CRYPTO 2003: 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003. Proceedings 23*, pages 463–481. Springer, 2003.
- [KPLG24] Elisabeth Krahmer, Peter Pessl, Georg Land, and Tim Güneysu. Correction fault attacks on randomized crystals-dilithium. *Cryptology ePrint Archive*, 2024.
- [Lyu09] Vadim Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *Advances in Cryptology – ASIACRYPT 2009*, volume 5912 of *Lecture Notes in Computer Science*, pages 598–616, Tokyo, Japan, December 6–10, 2009. Springer Berlin Heidelberg, Germany.
- [LZS⁺20] Yuejun Liu, Yongbin Zhou, Shuo Sun, Tianyu Wang, Rui Zhang, and Jingdian Ming. On the security of lattice-based fiat-shamir signatures in the presence of randomness leakage. *IEEE Transactions on Information Forensics and Security*, 16:1868–1879, 2020.
- [MMYSB19] Ricardo A Maronna, R. Douglas Martin, Victor J Yohai, and Matías Salibián-Barrera. *Robust Statistics: Theory and Methods (with R)*, volume 7 of *Wiley Series in Probability and Statistics*. Wiley, Newark, second edition. edition, 2019.
- [Moo77] Harry Richard Moore. *Robust regression using maximum-likelihood weighting and assuming Cauchy-distributed random error*. PhD thesis, Monterey, California. Naval Postgraduate School, 1977.
- [MPW21] Ben Marshall, Dan Page, and James Webb. Miracle: Micro-architectural leakage evaluation: A study of micro-architectural power leakage across many devices. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(1):175–220, Nov. 2021.
- [MWG⁺22] Soundes Marzougui, Nils Wisiol, Patrick Gersch, Juliane Krämer, and Jean-Pierre Seifert. Machine-learning side-channel attacks on the galactics constant-time implementation of bliss. In *Proceedings of the 17th International Conference on Availability, Relia-*

- bility and Security*, ARES '22, New York, NY, USA, 2022. Association for Computing Machinery.
- [Nat24] National Institute of Standards and Technology (NIST). Module-Lattice-Based Digital Signature Standard, 2024.
 - [PV17] Kostas Papagiannopoulos and Nikita Veshchikov. Mind the gap: Towards secure 1st-order masking in software. In Sylvain Guilley, editor, *Constructive Side-Channel Analysis and Secure Design*, pages 282–297, Cham, 2017. Springer International Publishing.
 - [QLZ⁺22] Zehua Qiao, Yuejun Liu, Yongbin Zhou, Jingdian Ming, Chengbin Jin, and Huizhong Li. Practical public template attacks on crystals-dilithium with randomness leakages. *IEEE Transactions on Information Forensics and Security*, 18:1–14, 2022.
 - [RJH⁺18] Prasanna Ravi, Mahabir Prasad Jhanwar, James Howe, Anupam Chattopadhyay, and Shivam Bhasin. Side-channel assisted existential forgery attack on Dilithium - A NIST PQC candidate. Cryptology ePrint Archive, Report 2018/821, 2018.
 - [SKL⁺20] Bo-Yeon Sim, Jihoon Kwon, Joohee Lee, Il-Ju Kim, Tae-Ho Lee, Jaeseung Han, Hyojin Yoon, Jihoon Cho, and Dong-Guk Han. Single-trace attacks on message encoding in lattice-based kems. *IEEE Access*, 8:183175–183191, 2020.
 - [SL03] George A. F. Seber and Alan J. Lee. *Linear regression analysis*. Wiley series in probability and statistics. Wiley-Interscience, Hoboken, NJ, 2. ed. edition, 2003.
 - [SLKG22] Hauke Steffen, Georg Land, Lucie Kogelheide, and Tim Güneysu. Breaking and protecting the crystal: Side-channel analysis of dilithium in hardware. Cryptology ePrint Archive, Paper 2022/1410, 2022.
 - [SPOG19] Tobias Schneider, Clara Paglialonga, Tobias Oder, and Tim Güneysu. Efficiently masking binomial sampling at arbitrary orders for lattice-based crypto. In *Public-Key Cryptography – PKC 2019: 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography, Beijing, China, April 14–17, 2019, Proceedings, Part II*, page 534–564, Berlin, Heidelberg, 2019. Springer-Verlag.
 - [TJSO14] Efthymios Tsakonas, Joakim Jaldén, Nicholas D Sidiropoulos, and Björn Ottersten. Convergence of the huber regression m-estimate in the presence of dense outliers. *IEEE Signal Processing Letters*, 21(10):1211–1214, 2014.
 - [Tro12] Joel A Tropp. User-friendly tail bounds for sums of random matrices. *Foundations of computational mathematics*, 12:389–434, 2012.
 - [UMB⁺23] Vincent Quentin Ulitzsch, Soundes Marzougui, Alexis Bagia, Mehdi Tibouchi, and Jean-Pierre Seifert. Loop aborts strike back: Defeating fault countermeasures in lattice signatures with ilp. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):367–392, 2023.

- [UMTS24] Vincent Quentin Ulitzsch, Soundes Marzougui, Mehdi Tibouchi, and Jean-Pierre Seifert. Profiling side-channel attacks on dilithium - A small bit-fiddling leak breaks it all. In Benjamin Smith and Huapeng Wu, editors, *SAC 2022: 29th Annual International Workshop on Selected Areas in Cryptography*, volume 13742 of *Lecture Notes in Computer Science*, pages 3–32, Windsor, Canada, August 24–26, 2024. Springer, Cham, Switzerland.
- [WLX⁺21] Tianyu Wang, Yuejun Liu, Jun Xu, Lei Hu, Yang Tao, and Yongbin Zhou. Integer lwe with non-subgaussian error and related attacks. In *International Conference on Information Security*, pages 3–25. Springer, 2021.

A Complete Power Trace Analysis of `impconvBA_rec64()`

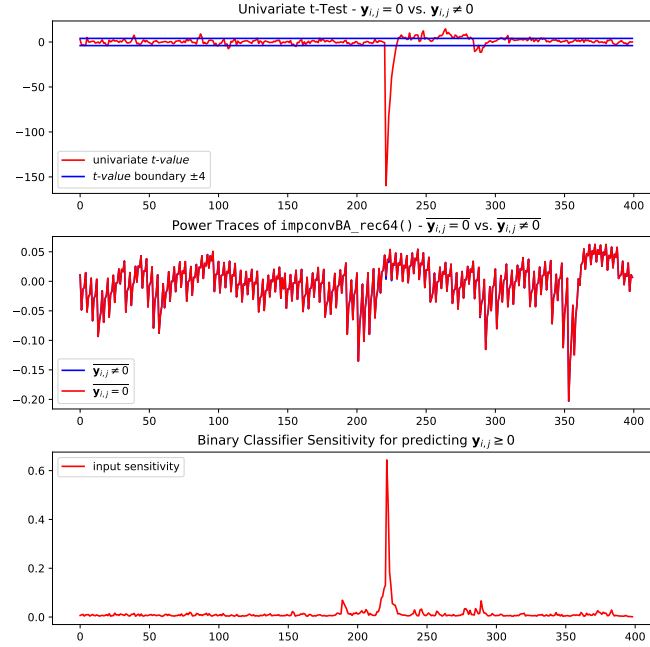


Fig. 9: The upper plot shows the t-test of the groups $y_{i,j} = 0$ (fixed) and $y_{i,j} \neq 0$ (random) computed on power traces collected during the *full* execution of the `impconvBA_rec64()` conversion function. Each group consists of 25000 power traces. Crossing the t-test plot's blue *t-value* boundaries of ± 4 corresponds to a p-value < 0.001 . In the center plot, averaged power traces of the same groups are displayed. The lower plot shows the input sensitivity analysis of a binary machine learning classifier trained to predict if power traces correspond to the conversion of coefficients $y_{i,j} \geq 0$. As is shown, the t-test discovered leakage aligns with the classifier's input sensitivity.

B Proof of Lemma 7.3

For convenience, we repeat the lemma.

Lemma (Smallest Eigenvalue of $\frac{1}{m}\mathbf{C}_A^T\mathbf{C}_A$ for Dilithium). Consider $A = \{i \mid \mathbf{y}_i = 0, \|\mathbf{C}_i\|_2 \leq \alpha\sqrt{m}\}$, the index set of error-free samples. Then, whenever $m \geq \frac{2^{10}n^2}{(1-p)^2}$ it holds with probability at least $1 - n \exp\left(-\frac{(1-p)m}{8n}\right)$ that the smallest eigenvalue of $\frac{1}{m}\mathbf{C}_A^T\mathbf{C}_A$ satisfies

$$\lambda_{\min}\left(\frac{1}{m}\mathbf{C}_A^T\mathbf{C}_A\right) \geq \frac{(1-p)\tau}{2n}.$$

Proof. We are interested in the eigenvalues of $\frac{1}{m}\mathbf{C}_A^T\mathbf{C}_A$ for the subset samples in $A = \{i \mid \mathbf{y}_i = 0, \|\mathbf{C}_i\|_2 \leq \alpha\sqrt{m}\}$ (zero-error and bounded norm). It has been shown by [DKM⁺25] that for *all* samples we have

$$\mathbb{E}\left[\frac{1}{m}\mathbf{C}^T\mathbf{C}\right] = \frac{\tau}{n}\mathbf{I}_n. \quad (7)$$

By design (Definition 7.1), the events $\{i \in A\}$ are i.i.d. with probability

$$\mathbb{P}(i \in A) = \mathbb{P}(\mathbf{y}_i = 0 \wedge \|\mathbf{C}_i\|_2 \leq \alpha\sqrt{m}) = \mathbb{P}(\mathbf{y}_i = 0)\mathbb{P}(\|\mathbf{C}_i\|_2 \leq \alpha\sqrt{m}) \quad (8)$$

As the norm of the rows of $\mathbf{C}$ is constant with $\|\mathbf{C}_i\|_2 = \sqrt{\tau}$, we have

$$\|\mathbf{C}_i\|_2 \leq \alpha\sqrt{m} = \frac{(1-p)\sqrt{\tau m}}{2^5 n} \implies m \geq \left(\frac{2^5 n}{1-p}\right)^2$$

implying $\mathbb{P}(\|\mathbf{C}_i\|_2 \leq \alpha\sqrt{m}) = 1$ whenever $m \geq \frac{2^{10}n^2}{(1-p)^2}$ (which we assume for the sequel of this proof). In that case, $\mathbb{P}(i \in A) = \mathbb{P}(\mathbf{y}_i = 0) = (1-p)$ which together with Equation (7) implies

$$\mathbb{E}\left[\frac{1}{m}\mathbf{C}_A^T\mathbf{C}_A\right] = \frac{(1-p)\tau}{n}\mathbf{I}_n. \quad (9)$$

Hence, all eigenvalues of $\frac{1}{m}\mathbf{C}_A^T\mathbf{C}_A$ will concentrate around $\frac{(1-p)\tau}{n}$ for m large enough. It remains to quantify the convergence speed. We need to bound the smallest eigenvalue of $\frac{1}{m}\mathbf{C}_A^T\mathbf{C}_A$ from below. In accordance with [DKM⁺25], we first make use of the fact that this matrix can equally be expressed as the sum of outer products $\mathbf{C}^T\mathbf{C} = \sum_{i=1}^m \mathbf{C}_i\mathbf{C}_i^T$. From this we have

$$\frac{1}{m}\mathbf{C}_A^T\mathbf{C}_A = \sum_{i=1}^m \underbrace{\left(\frac{1}{m}\mathbf{C}_i\mathbf{C}_i^T \cdot \mathbb{1}_{[i \in A]}\right)}_{\mathbf{S}_i}. \quad (10)$$

The matrices $\mathbf{S}_i$ are independent (rows $\mathbf{C}_i$ are i.i.d. according to Definition 7.1) bounded and symmetric, thus we can apply a Matrix Chernoff bound [Tro12]. For this, we investigate the spectrum of $\mathbf{S}_i$. Since $\mathbf{S}_i$ is a rank one matrix, $n - 1$ eigenvalues are 0, and the remaining eigenvalue can be bounded using the trace $\lambda(\mathbf{S}_i) = \text{Tr}[\mathbf{S}_i] = \sum_{j=1}^n \frac{1}{m} \mathbf{C}_{ij}^2 = \frac{\tau}{m}$ (or $= 0$ if $i \notin A$), i.e.,

$$\lambda_{\min}(\mathbf{S}_i) = 0 \quad \text{and} \quad \lambda_{\max}(\mathbf{S}_i) \leq \frac{\tau}{m} \quad (:= R) .$$

We combine Equations (9) and (10) and provided that $m \geq \frac{2^{10} n^2}{(1-p)^2}$ we have

$$\begin{aligned} \mu_{\min} &:= \lambda_{\min} \left(\sum_{i=1}^m \mathbb{E}[\mathbf{S}_i] \right) \stackrel{(10)}{=} \lambda_{\min} \left(\frac{1}{m} \mathbf{C}_A^T \mathbf{C}_A \right) \stackrel{(9)}{=} (1-p) \lambda_{\min} \left(\mathbb{E} \left[\frac{1}{m} \mathbf{C}^T \mathbf{C} \right] \right) \\ &= (1-p) \lambda_{\min} \left(\frac{\tau}{n} \mathbf{I}_n \right) = (1-p) \frac{\tau}{n} . \end{aligned}$$

Then the Matrix Chernoff bound⁶ states that for $c_1 \in [0, 1)$

$$\mathbb{P} \left(\lambda_{\min} \left(\sum_{i=1}^m \mathbf{S}_i \right) \leq (1 - c_1) \mu_{\min} \right) \leq n \exp \left(- \frac{c_1^2 \mu_{\min}}{2R} \right) .$$

We set $c_1 = \frac{1}{2}$ and obtain

$$\mathbb{P} \left(\lambda_{\min} \left(\frac{1}{m} \mathbf{C}_A^T \mathbf{C}_A \right) \leq \frac{(1-p)\tau}{2n} \right) \leq n \exp \left(- \frac{(1-p)m}{8n} \right) .$$

That concludes the proof. $\square$

C Proof of Lemma 7.4

For convenience, we repeat the lemma.

Lemma (Gradient of the Loss Function for Dilithium). The norm of the gradient of the empirical Huber loss at the true value of $\mathbf{s}$ is bounded with probability $1 - \frac{1}{8}$

$$\|\nabla L_m(\mathbf{s})\|_2 \leq \frac{8\delta\sqrt{\tau}}{\sqrt{m}} .$$

Proof. We need to bound $\|\nabla L_m(\mathbf{s})\|_2$, the norm of the gradient (w.r.t. $\mathbf{s}'$) at the true secret $\mathbf{s}$ for the Huber loss function ρ_δ , given in Definition 4.2.

We start by deriving an upper bound on the expectation of the gradient norm (w.r.t. the distribution of $\mathbf{C}_i$ and $\mathbf{y}_i$). By linearity, we have

$$\mathbb{E} \|\nabla L_m(\mathbf{s})\|_2 = \mathbb{E} \left[\left\| \frac{1}{m} \sum_{i=1}^m \nabla \rho_\delta(\mathbf{z}_i - \mathbf{C}_i^T \mathbf{s}) \right\|_2 \right] . \quad (11)$$

⁶ More precisely we utilize Remark 5.3 in [Tro12], a simplifying corollary of the Matrix Chernoff bound II (Corollary 5.2 in [Tro12]).

We invoke $\mathbf{z}_i - \mathbf{C}_i^T \mathbf{s} = \mathbf{y}_i$ for the true regressor $\mathbf{s}$, and by applying Jensen's inequality we have $\mathbb{E}[X] \leq \sqrt{\mathbb{E}[X^2]}$, therefore

$$= \mathbb{E} \left[\left\| \frac{1}{m} \sum_{i=1}^m \rho'_\delta(\mathbf{y}_i) \mathbf{C}_i \right\|_2 \right] \quad (12)$$

$$\leq \sqrt{\mathbb{E} \left[\left\| \frac{1}{m} \sum_{i=1}^m \rho'_\delta(\mathbf{y}_i) \mathbf{C}_i \right\|_2^2 \right]} \quad (13)$$

$$= \sqrt{\frac{1}{m^2} \mathbb{E} \left[\left\| \sum_{i=1}^m \rho'_\delta(\mathbf{y}_i) \mathbf{C}_i \right\|_2^2 \right]} . \quad (14)$$

Next, we expand the square inside the expectation

$$\begin{aligned} \mathbb{E} \left[\left\| \sum_{i=1}^m \rho'_\delta(\mathbf{y}_i) \mathbf{C}_i \right\|_2^2 \right] &= \mathbb{E} \left[\left(\sum_{i=1}^m \rho'_\delta(\mathbf{y}_i) \mathbf{C}_i \right)^T \left(\sum_{j=1}^m \rho'_\delta(\mathbf{y}_j) \mathbf{C}_j \right) \right] \\ &= \mathbb{E} \left[\sum_{i=1}^m \sum_{j=1}^m \rho'_\delta(\mathbf{y}_i) \rho'_\delta(\mathbf{y}_j) \mathbf{C}_i^T \mathbf{C}_j \right] . \end{aligned}$$

Since $\rho'_\delta(\mathbf{y}_i)$ and $\mathbf{C}_i$ are independent by assumption we get

$$= \sum_{i=1}^m \mathbb{E}[\rho'_\delta(\mathbf{y}_i)^2] \mathbb{E}[\|\mathbf{C}_i\|_2^2] + \sum_{i \neq j} \mathbb{E}[\rho'_\delta(\mathbf{y}_i)] \mathbb{E}[\rho'_\delta(\mathbf{y}_j)] \mathbb{E}[\mathbf{C}_i^T \mathbf{C}_j] .$$

We have $\mathbb{E}[\mathbf{C}_i^T \mathbf{C}_j] = \mathbb{E}[\mathbf{C}_i^T] \mathbb{E}[\mathbf{C}_j] = 0$ for $i \neq j$ by independence of $\mathbf{C}_i$ and $\mathbf{C}_j$ (whenever $i \neq j$) and the fact that $\mathbb{E}[\mathbf{C}_i] = \mathbf{0}$, so the second sum evaluates to zero. On top, $\mathbf{C}_i$ is a n dimensional vector where each coefficient is ± 1 with probability $\frac{\tau}{n}$. We conjecture that $\mathbb{E}[\|\mathbf{C}_i\|_2^2] = \sum_{j=1}^n \frac{\tau}{n} = \tau$

$$\frac{1}{m^2} \mathbb{E} \left[\left\| \sum_{i=1}^m \rho'_\delta(\mathbf{y}_i) \mathbf{C}_i \right\|_2^2 \right] = \frac{1}{m^2} \sum_{i=1}^m \underbrace{\mathbb{E}[\rho'_\delta(\mathbf{y}_i)^2]}_{\leq \delta^2} \underbrace{\mathbb{E}[\|\mathbf{C}_i\|_2^2]}_{=\tau} \leq \frac{\delta^2 \tau}{m} .$$

and together with Equation (14) the expectation of the gradient norm is bounded

$$\mathbb{E} \|\nabla L_m(\mathbf{s})\|_2 \leq \frac{\delta \sqrt{\tau}}{\sqrt{m}} . \quad (15)$$

Next, we use the Markov inequality to bound its probability.

$$\begin{aligned} \mathbb{P} \left(\|\nabla L_m(\mathbf{s})\|_2 < \frac{c_2 \delta \sqrt{\tau}}{\sqrt{m}} \right) &\geq \mathbb{P} (\|\nabla L_m(\mathbf{s})\|_2 < c_2 \mathbb{E} \|\nabla L_m(\mathbf{s})\|_2) \\ &\geq 1 - \frac{1}{c_2} \end{aligned}$$

We choose $c_2 = \frac{1}{8}$ to have a decently high success probability and obtain

$$\|\nabla L_m(\mathbf{s})\|_2 < \frac{8\delta\sqrt{\tau}}{\sqrt{m}}.$$

This concludes the proof.

□